

- Administración del sistema de usuarios, seguridad, código y tablas de constantes, etcétera.
- Procesamiento automático de ventas sin conexión cuando fallen los componentes externos.
- Transacciones en tiempo real, basadas en estándares industriales, con sistemas de terceras partes, que incluye los servicios de inventario, contabilidad, recursos humanos, cálculo de impuestos, y autorización de pagos.
- Definición y ejecución de reglas del negocio “conectables” en puntos comunes y fijos de los escenarios de procesamiento.
- ...

Otros requisitos y restricciones

Abarca las restricciones de diseño, facilidad de uso, fiabilidad, rendimiento, soporte, empaquetado, documentación, etcétera: Diríjase a la Especificación Complementaria y a los casos de uso.

7.5. Comentario: Visión

¿Estamos solucionando el mismo problema?
¿El problema correcto?

El enunciado del problema

Durante el trabajo de requisitos en la fase de inicio, hay que colaborar para definir un enunciado del problema conciso; reducirá la posibilidad de que el personal involucrado esté intentando solucionar problemas ligeramente diferentes, y normalmente, se crea rápido. Algunas veces, este trabajo revela diferencias de opinión fundamentales en lo que las partes están tratando de conseguir.

Una alternativa al simple texto es el formato de tabla propuesto en la plantilla del RUP para el enunciado del problema:

El problema de	...
afecta	...
el impacto de lo cual es	...
una solución de éxito sería	...

Los objetivos de alto nivel y problemas claves del personal involucrado

Esta tabla resume los objetivos y problemas a un nivel más alto que los casos de uso de nivel de tarea, y muestra importantes objetivos no funcionales y de calidad que podrían pertenecer a un caso de uso o abarcar a muchos, tales como:

- *Necesitamos procesamiento de ventas tolerante a fallos.*
- *Necesitamos la capacidad de adaptar las reglas del negocio.*

¿Cuáles son los problemas y objetivos esenciales?

Es típico que el personal involucrado exprese sus objetivos en base a soluciones que imaginan, como “Necesitamos un programador a tiempo completo que adapte las reglas del negocio cuando las cambiemos”. Las soluciones algunas veces son inteligentes, porque comprenden bien el dominio del problema y las opciones. Pero, algunas veces, el personal involucrado llega a soluciones que no son las más apropiadas o que no tratan el problema subyacente más importante y esencial.

Por tanto, el analista del sistema, necesita estudiar la cadena de problemas y objetivos —como se presentó en el capítulo anterior sobre los objetivos y casos de uso— para aprender los problemas subyacentes, y su importancia e impacto relativos, y de este modo priorizar y solucionar las cuestiones más relevantes con una solución ingeniosa.

Métodos para facilitar la idea del grupo

Aunque queda fuera del alcance de esta discusión, especialmente durante actividades tales como definición de problemas de alto nivel e identificación de objetivos, tiene lugar el trabajo de investigación creativo en grupo. Presentamos algunas técnicas útiles de ayuda al grupo para el descubrimiento de los problemas y objetivos esenciales y para ayudar a la generación de ideas y su priorización: mapas mentales, diagramas causa-efecto (*fish-bone*), diagramas pareto, “tormenta de ideas” (*brainstorming*), multivotación, votación por puntos (*dot voting*), técnica de grupo nominal, “escribir ideas en silencio” (*brainwriting*) y diagrama de afinidad. Estúdielos en la web. Prefiero aplicar varios de ellos durante el mismo taller, para descubrir problemas típicos y requisitos desde distintos ángulos.

Características del sistema—requisitos funcionales

Los casos de uso no son necesariamente el único modo para expresar los requisitos funcionales, por los siguientes motivos:

- Son detallados. A menudo, el personal involucrado quiere un resumen breve que identifique las funciones más destacables.
- ¿Y si listamos simplemente los nombres de los casos de uso (*Procesar Venta, Gestionar Devoluciones,...*) para resumir la funcionalidad? Primero, la lista podría ser todavía demasiado extensa. Además, los nombres pueden ocultar funcionalidad interesante que el personal involucrado realmente quiere conocer; esto es, el nivel de granularidad puede oscurecer funciones importantes. Por ejemplo, suponga que la descripción de la funcionalidad de la autorización automática de pago está contenida en el caso de uso *Procesar Venta*. Un lector de la lista de los nombres de los casos de uso no puede decir si el sistema realizará autorizaciones de pagos. Es más, uno podría desear agrupar un conjunto de casos de uso en una característica (por simplicidad), como *Administración del sistema para usuarios, seguridad, código y tablas de constantes, etcétera*.

- Algunas funcionalidades importantes, como es lógico, se expresan como sentencias cortas que no se corresponden de manera conveniente con los nombres de los casos de uso u objetivos de nivel de Procesos del Negocio Elementales (EBP). Podrían abarcar o ser ortogonales a los casos de uso. Por ejemplo, durante el primer taller de requisitos de NuevaEra, algunos podrían decir “el sistema debería ser capaz de llevar a cabo transacciones con sistemas de terceras partes de contabilidad, inventario y cálculo de impuestos”. Esta sentencia sobre la funcionalidad no representa ningún caso de uso en particular, pero es una manera cómoda y concisa para expresar, recoger y comunicar las características.
- Una variación más drástica del último punto es la siguiente. Algunas aplicaciones requieren ante todo una descripción de la funcionalidad como características; los casos de uso no se ajustan de manera natural. Esto es común, por ejemplo, en productos *middleware*, como los servidores de aplicaciones —los casos de uso no están motivados realmente—. Suponga que el equipo está considerando la próxima versión. Durante un debate de los requisitos, la gente (como los de marketing) dirá: “La siguiente versión necesita soportar *bean* entidad EJB2.0”. Los requisitos se conciben inicialmente en términos de una lista de características, no de casos de uso.

Por tanto, un modo alternativo, complementario para expresar las funciones del sistema, es mediante las **características**, o más concretamente en este contexto, **características del sistema**, que son sentencias concisas, de alto nivel, que resumen las funciones del sistema. Más formalmente, en el UP, un **característica del sistema** es “un servicio observable externamente proporcionado por el sistema que cumple directamente una necesidad del personal involucrado” [Kruchten00].

Las características son cosas que un sistema puede *hacer*. Deberían pasar este test lingüístico:
El sistema deberá hacer <característicaX>.

Por ejemplo:

El sistema deberá hacer la autorización del pago.

Recordemos que la Visión podría utilizarse como contrato formal o informal entre los desarrolladores y la empresa. Las características del sistema constituyen un mecanismo para resumir, en este contrato, lo que el sistema *hará*. Esto es complementario a los casos de uso, ya que las características son concisas.

Las características tienen que contrastarse con varios tipos de requisitos no funcionales y restricciones, por ejemplo: “*El sistema debe ejecutarse bajo Linux, debe tener disponibilidad de 24/7 y debe tener una interfaz con pantalla táctil*”. Nótese que falla el test lingüístico.

A veces, la propiedad de que sea “un servicio observable externamente...” es difícil de resolver. Por ejemplo, ¿debería ser lo siguiente una característica del sistema?:

El sistema hará las transacciones con sistemas de terceras partes de contabilidad, inventario, recursos humanos y cálculo de impuestos.

Se trata de un tipo de comportamiento, y probablemente de interés para el personal involucrado, pero la colaboración en sí misma podría no ser visible externamente, dependiendo del margen de tiempo, lo cerca que se esté y de dónde se mire. Téngalo en cuenta —rara vez merece la pena que se preocupe por cuestiones de clasificación de poca granularidad—.

Por último, podemos observar que la mayoría de las características del sistema se expresarán de manera detallada en el texto de los casos de uso.

Notación y organización

Ante todo, son importantes las descripciones breves de alto nivel. Uno debería ser capaz de leer la lista de características del sistema rápidamente.

No es necesario incluir la frase canónica “El sistema deberá hacer...” o una frase variante, aunque es lo normal.

A continuación, presentamos un ejemplo de características de alto nivel, para un proyecto multi-sistema amplio, del cual el PDV es sólo un elemento:

Las características principales incluyen:

- *Servicios PDV.*
- *Gestión de inventario.*
- *Compras basadas en la web.*
- ...

Es habitual organizar una jerarquía de dos niveles de características del sistema. Pero en el documento de Visión, más de dos niveles nos llevaría a un detalle excesivo; el sentido de las características del sistema en la Visión es resumir la funcionalidad, no descomponerla en una larga lista de elementos de grano fino. Un ejemplo razonable en cuanto a detalle:

Las características principales incluyen:

- *Servicios PDV:*
 - *Capturar ventas.*
 - *Autorización de pago.*
 - ...
- *Gestión de inventario:*
 - *Reordenación automática.*
 - ...

Algunas veces, estas características de segundo nivel son básicamente equivalentes a los nombres de los casos de uso (u objetivos de nivel de usuario), pero no es imprescindible; las características son un modo alternativo para resumir la funcionalidad. No obstante, la mayoría de las características del sistema se expresarán de manera detallada en el texto de los casos de uso.

¿Cuántas características del sistema debería contener la Visión?

Sugerencia

Es deseable un documento de Visión con menos de 50 características. Si hay más, considere la posibilidad de agrupar y abstraer las características.

Otros requisitos en la Visión

En la Visión, las características del sistema resumen brevemente los requisitos funcionales, que se expresan en detalle en los casos de uso. De igual modo, la Visión *puede* resumir otros requisitos (por ejemplo, fiabilidad y facilidad de uso) que se detallan en la sección de *Requisitos Especiales* de los casos de uso, y en la Especificación Complementaria (SS, *Supplementary Specification*). Sin embargo, hay peligro de duplicaciones inútiles. Por ejemplo, el producto RUP proporciona plantillas para la Visión y SS que contienen secciones idénticas o similares para otros requisitos como facilidad de uso, fiabilidad, rendimiento, etcétera. Tal duplicación es inevitablemente difícil de mantener. Además, el nivel de detalle de las secciones similares (por ejemplo, rendimiento) en la Visión y SS, necesita que sea bastante parecido para que tenga sentido; es decir, las descripciones de otros requisitos “detalladas” y “esenciales”, tienden a ser prácticamente iguales.

Sugerencia

Para otros requisitos, evite su duplicación incluyéndolo de forma idéntica o casi idéntica, tanto en la Visión como en la Especificación Complementaria (SS) —y en los casos de uso—. Más bien, recójalos sólo en la SS o los casos de uso (si son específicos de los casos de uso). En la Visión dirija al lector a las secciones de éstos donde puede encontrar estos otros requisitos.

Visión, características o casos de uso—¿qué va primero?

No es útil ser estricto en el orden de algunos artefactos. Durante la colaboración para crear diferentes artefactos de requisitos, se produce una sinergia en la que el trabajo con uno influye y ayuda a clarificar los otros. Sin embargo, la secuencia recomendada es:

- 1. Escribir un primer borrador breve de la Visión.
- 2. Identificar los objetivos de usuario y los casos de uso de apoyo.
- 3. Escribir algunos casos de uso y comenzar la Especificación Complementaria.
- 4. Refinar la Visión, resumiendo la información a partir de éstos.

7.6. Ejemplo NuevaEra: un Glosario (Parcial)

Glosario			
Historia de revisiones			
Versión	Fecha	Descripción	Autor
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

Definiciones

Término	Definición e Información	Alias
artículo	Un artículo o servicio en venta.	
autorización de pago	Validación llevada a cabo por un servicio externo de autorización de pago, que hará o garantizará el pago al vendedor.	
solicitud de autorización de pago	Un compuesto de elementos enviados electrónicamente a un servicio de autorización, normalmente como un array de caracteres. Los elementos comprenden: ID de la tienda, número de cuenta del cliente, cantidad y fecha.	
UPC	Código de 12 dígitos que identifica un artículo. Normalmente se representa mediante un código de barras en los artículos. Diríjase a http://www.uc-council.org para ver más detalles.	Código de Producto Universal
...	...	...

7.7. Comentario: Glosario (Diccionario de Datos)

En su forma más simple, el **Glosario** es una lista de los términos relevantes y sus definiciones. Es sorprendentemente habitual que un término, frecuentemente técnico o propio del dominio, se utilice de forma ligeramente distinta por diferentes personas involucradas; esto tiene que resolverse para reducir los problemas de comunicación y los requisitos ambiguos.

Sugerencia

Comience el Glosario cuanto antes. Recuerdo una experiencia trabajando con expertos en simulación, en la que descubrimos que la aparentemente inocua, pero importante, palabra “celda” tenía significados diversos y escurridizos entre los miembros del grupo.

El objetivo no es recoger todos los posibles términos, sino aquellos que no están claros, son ambiguos o que requieren algún tipo de elaboración relevante, como el formato de la información o las reglas de validación.

El Glosario como diccionario de datos

En el UP, el Glosario también juega el rol de **diccionario de datos**, un documento que recoge los datos sobre los datos, es decir, **metadatos**. Durante la fase de inicio, el glosario debe ser un documento sencillo de términos y descripciones. Durante la elaboración, podría ampliarse a un diccionario de datos.

Los atributos de los términos podrían contener:

- Alias.
- Descripción.
- Formato (tipo, longitud, unidad).
- Relaciones con otros elementos.

- Rango de valores.
- Reglas de validación.

Observe que el rango de valores y las reglas de validación del Glosario constituyen requisitos con implicaciones en el comportamiento del sistema.

Unidades

Como subraya Martin Fowler en *Analysis Patterns* [Fowler96], se deben tener en cuenta las unidades (moneda, medidas,...), especialmente en esta era de aplicaciones software internacionalizadas. Por ejemplo, en el sistema NuevaEra, que, con un poco de suerte, se venderá a muchos clientes en diferentes países, el *precio* no puede ser sólo un simple número. Debe estar en una unidad de *Dinero* o *Moneda* que permita la variación de monedas.

Términos compuestos

El Glosario no está destinado sólo a términos atómicos como el “precio del artículo”. Puede y debe incluir términos compuestos, como “venta” (que incluye otros elementos, tales como la fecha y la ubicación), y alias utilizados para describir una colección de datos que se transmiten entre los actores en los casos de uso. Por ejemplo, en el caso de uso *Procesar Venta*, considere la siguiente sentencia:

El Sistema envía una *solicitud de autorización de pago* a un Servicio externo de Autorización de Pago, y solicita la aprobación del pago.

La “solicitud de autorización del pago” es un alias para un agregado de datos, que tiene que explicarse en el Glosario.

7.8. Especificaciones fiables: ¿Un Oxímoron²?

Los requisitos escritos pueden dar la impresión de que se han entendido los requisitos reales y están bien definidos, y pueden utilizarse (pronto) para estimar y planificar el proyecto de manera fiable. Esta impresión es más intensa en los desarrolladores no programadores; los programadores conocen a partir de experiencias dolorosas lo poco fiable que es. Esto justifica en parte de la motivación de la cita de Goethe del comienzo del capítulo.

Lo que realmente importa es construir software que pase las pruebas de aceptación definidas por los usuarios y el resto de personas implicadas en el proyecto, y que satisfaga sus verdaderos objetivos (que, a menudo, no se descubren hasta que están evaluando o trabajando con el software).

² *N. del T.*: Figura en la que el adjetivo que califica un nombre contradice la esencia del nombre que modifica, como en “nieve ardiente”.

Es importante escribir como ejercicio la Visión y Especificación Complementaria para poner en claro una primera aproximación de lo que se quiere, la motivación del producto y como repositorio para las grandes ideas. Pero no son —ni ningún artefacto de requisitos— una especificación fiable. Sólo escribiendo código, probándolo, obteniendo retroalimentación, manteniendo una estrecha colaboración con los usuarios y los clientes y adaptando, se da realmente en el blanco.

Esto no es una llamada a abandonar el análisis y la reflexión e ir directamente a escribir código, sino una sugerencia para tratar los requisitos escritos de una manera ligera, y continuamente involucrar a los usuarios —de hecho, debía ser diariamente—.

7.9. Artefactos disponibles en el sitio web del proyecto

Puesto que esto es un libro, estos ejemplos y los anteriores casos de uso tienen un aspecto estático y, quizás orientado al papel. Sin embargo, deberían ser artefactos digitales recogidos sólo on-line a través del sitio web del proyecto. Y, en lugar de ser documentos estáticos planos, podrían contener hipervínculos, o guardarse en herramientas distintas a un procesador de texto u hoja de cálculo. Por ejemplo, el Glosario podría almacenarse en una tabla de una base de datos.

7.10. ¿Poco UML durante la fase de inicio?

El objetivo de la fase de inicio es recopilar sólo la información suficiente para establecer una visión común, decidir si es viable avanzar y si merece la pena una investigación seria del proyecto en la fase de elaboración. Como tal, a menudo, no son necesarios más diagramas que los simples diagramas de casos de uso UML. Durante la fase de inicio se hace hincapié en entender el alcance básico y el 10% de los requisitos, expresados textualmente. En la práctica y, por tanto, en esta presentación, la mayoría de los diagramas UML se realizarán en la siguiente fase —elaboración—.

7.11. Otros artefactos de requisitos en el UP

Como en el anterior capítulo de casos de uso, la Tabla 7.1 resume una muestra de los artefactos y su evolución temporal. Todos los artefactos de requisitos se comienzan en la fase de inicio, se trabaja sobre ellos sobre todo durante la elaboración.

Inicio

No debería ser el caso que estos artefactos de requisitos se terminen en la fase de inicio. De hecho, apenas se habrán empezado.

El personal involucrado necesita decidir si merece la pena llevar a cabo una investigación seria en el proyecto; esta investigación tiene lugar durante la elaboración, no la fase de inicio. Durante la fase de inicio, la Visión resume la idea del proyecto de manera que ayude, a los que tiene que tomar la decisión, a determinar si merece la pena continuar, y dónde comenzar.

Tabla 7.1. Muestra de los artefactos UP y evolución temporal. c – comenzar; r – refinar.

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso <i>Visión</i> <i>Especificación Complementaria</i> <i>Glosario</i>	c	r		
		c	r		
		c	r		
		c	r		
Diseño	Modelo de Diseño Documento de Arquitectura SW Modelo de Datos		c	r	
			c		
			c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Puesto que la mayoría del trabajo de requisitos ocurre durante la elaboración, la Especificación Complementaria debería desarrollarse ligeramente durante la fase de inicio, resaltando los atributos de calidad relevantes (por ejemplo, el PDV NuevaEra debe recuperarse cuando fallen los servicios externos) que muestren mayores riesgos y retos.

La información de entrada a estos artefactos podría generarse durante un taller de requisitos de la fase de inicio, tanto mediante la consideración explícita de estas cuestiones, como indirectamente, por medio del análisis de casos de uso. En el taller no se escribirá un borrador legible de los artefactos, sino que lo elaborará después el analista del sistema.

Elaboración

A través de las iteraciones de la elaboración, se refina la “visión” y la Visión, en base a la retroalimentación a partir de la construcción incremental de partes del sistema, la adaptación, y varios talleres de requisitos a lo largo de varias iteraciones de desarrollo.

Mediante la investigación de requisitos continuada y el desarrollo iterativo, los otros requisitos llegarán a verse más claros y se recogerán en la SS. Los atributos de calidad (por ejemplo, fiabilidad) identificados en la SS, serán claves para dar forma al núcleo de la arquitectura que se diseña y programa durante la elaboración. También pueden ser factores de riesgo claves que influyen en lo que se va a trabajar en las primeras iteraciones. Por ejemplo, el requisito de calidad del PDV NuevaEra de recuperación en el lado del cliente si fallan los componentes externos, se explorará durante la elaboración.

La mayoría de los términos se descubren y elaboran en el Glosario durante esta fase.

Llegando al final de la elaboración, es viable contar con casos de uso, una Especificación Complementaria y un documento de Visión que, de manera razonable, reflejan las características estabilizadas más importantes y otros requisitos que se deben completar para entregar el producto. Sin embargo, la Especificación Complementaria y la

Visión no se congelan o “se dan por concluidas” como especificaciones fijas; la adaptación —no la rigidez— es un valor central del desarrollo iterativo y el UP.

Aclaremos el comentario “dar por concluido y congelar”: es perfectamente razonable —al final de la elaboración— llegar a un acuerdo con el personal involucrado sobre lo que se hará en lo que queda del proyecto, y comprometerse (quizás contractualmente) considerando los requisitos y la planificación. En algunos puntos (el final de la elaboración, en el UP), necesitamos una idea fiable sobre “qué, cuánto y cómo”. En este sentido, es normal, y se espera, un acuerdo formal sobre los requisitos. También es necesario contar con un proceso de control de cambios (una de las mejores prácticas explícitas del UP) de forma que se tienen en cuenta y se aprueban, formalmente, los cambios en los requisitos, en lugar de cambios caóticos e incontrolados.

Más bien, el comentario “dar por concluido y congelar” implica varias ideas:

- En el desarrollo iterativo y el UP, se entiende que no importa cuanta atención merecida se presta a la especificación de requisitos, algún cambio es inevitable, y se debería aceptar. Este cambio podría ser una mejora oportunista que incorpora una novedad importante en el sistema que confiere a sus propietarios ventaja competitiva, o un cambio debido a una visión mejorada.
- En el desarrollo iterativo, es un valor central contar con un compromiso continuo por parte del personal involucrado para evaluar, proporcionar retroalimentación, y dirigir el proyecto como realmente quieren. No beneficia al personal involucrado el “lavarse las manos” de un compromiso solícito, dando por concluido un conjunto de requisitos congelados y esperando el producto final, porque raramente obtendrán lo que realmente necesitan.

Construcción

En la construcción, los requisitos importantes —tanto funcionales como de otro tipo— deberían estar estabilizados —no terminados—, aunque dispuestos a cambios menores. Por tanto, no es probable que la SS y la Visión experimenten muchos cambios en esta fase.

7.12. Lecturas adicionales

Los documentos como el de Visión y Especificaciones Complementarias no son nuevos. Se utilizan en muchos proyectos y se describen en muchos libros de requisitos. Muchos de estos libros asumen de manera implícita la actitud del proceso en cascada, en el que el objetivo es obtener los requisitos detallados y correctos al principio, y llegar a un compromiso, antes de pasar al diseño y la implementación. En este sentido, la descripción tradicional que presentan no es útil aunque, por otra parte, proporcionan buenos consejos sobre posibles secciones y sus contenidos.

La mayoría de los libros sobre arquitectura software incluyen una discusión sobre el análisis de requisitos para los requisitos de calidad de la aplicación, puesto que estos requisitos de calidad tienden a influenciar fuertemente el diseño de la arquitectura. Un ejemplo es *Software Architecture in Practice* [BCK98].

Las reglas de negocio se tratan de manera exhaustiva en *The Business Rule Book* [Ross97]. El libro presenta una teoría amplia, profunda y muy bien desarrollada de las reglas del negocio, pero el método no se conecta de manera adecuada con otras técnicas de requisitos modernas, como los casos de uso o el desarrollo iterativo.

7.13. Artefactos UP y contexto del proceso

La Figura 7.1 muestra la influencia entre los artefactos, destacando los documentos de Visión, Especificación Complementaria y el Glosario.

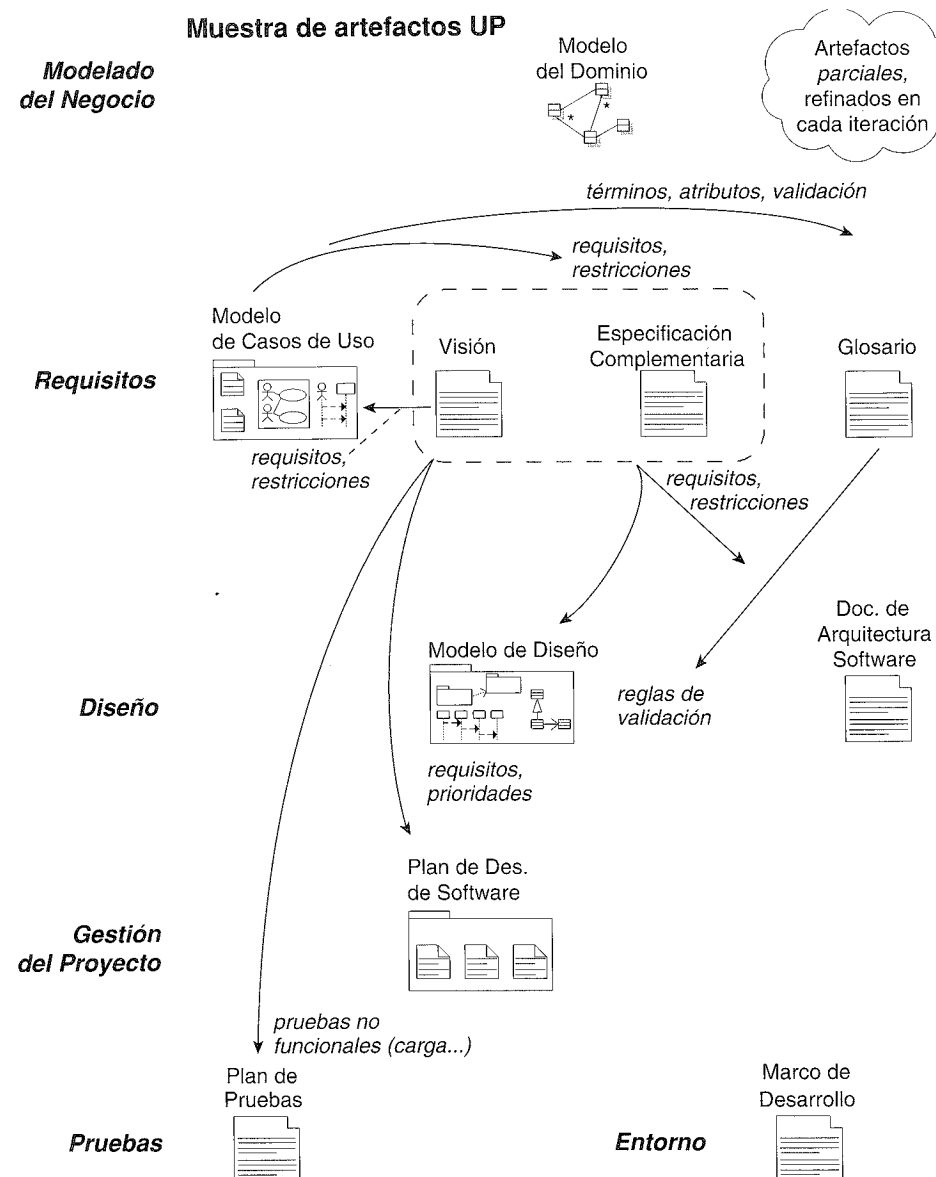


Figura 7.1. Muestra de la influencia entre los artefactos UP.

En el UP, el trabajo de Visión y Especificación Complementaria es una actividad de la disciplina de requisitos que podría iniciarse durante un taller de requisitos, junto con el análisis de casos de uso. La Figura 7.2 proporciona algunos consejos sobre el momento y el lugar para llevar a cabo este trabajo.

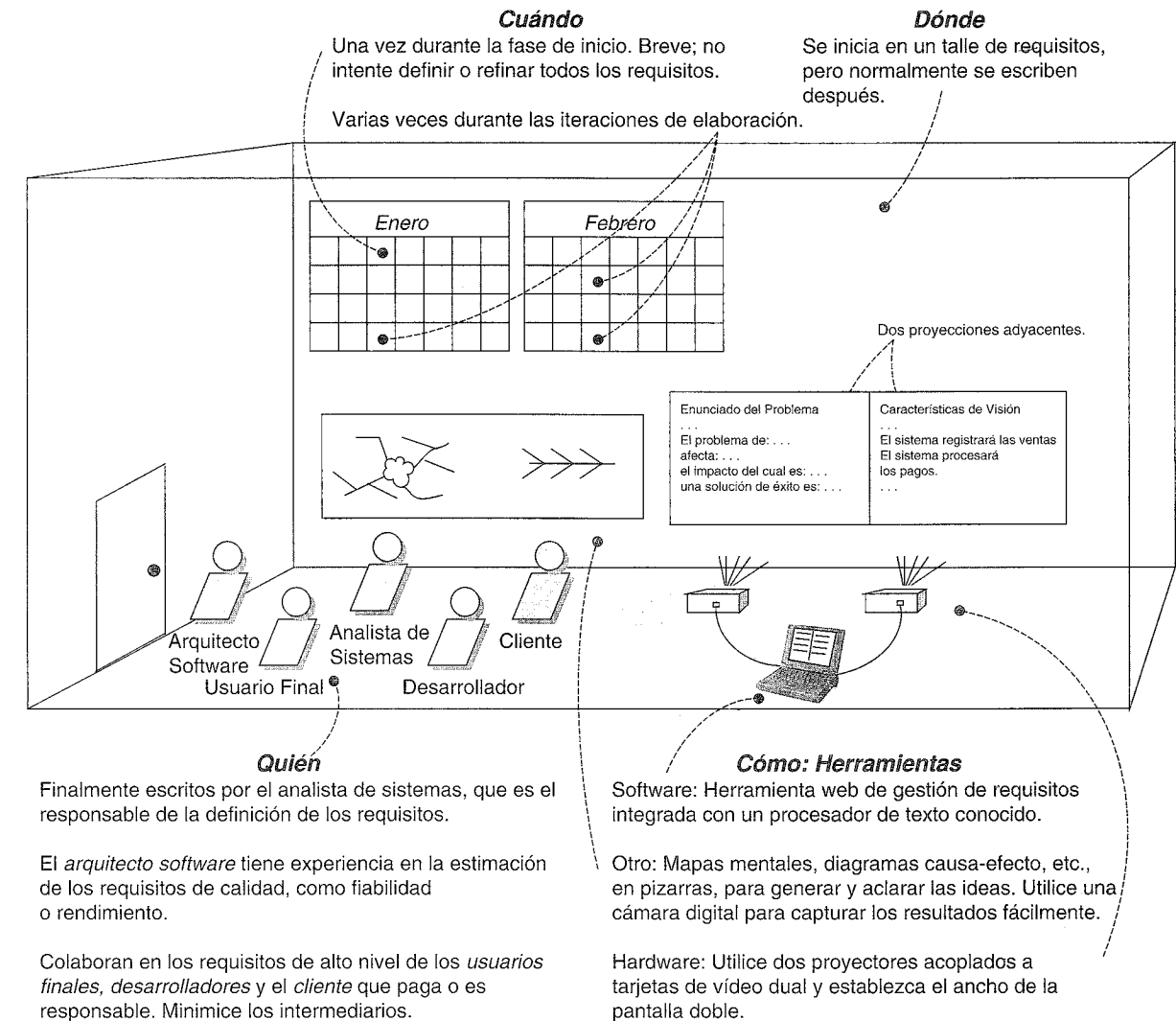


Figura 7.2. Proceso y establecimiento del contexto.

Capítulo 8

DEL INICIO
A LA ELABORACIÓN

Lo duro y rígido se rompe. Lo flexible prevalece.

Tao Te Ching

Objetivos

- Definir la etapa de elaboración.
- Motivar los siguientes capítulos de esta sección.

Introducción

La elaboración es la serie inicial de iteraciones durante la que:

- se descubren y estabilizan la mayoría de los requisitos
- se reducen o eliminan los riesgos importantes
- se implementan y prueban los elementos básicos de la arquitectura

Rara vez, la arquitectura no es un riesgo —por ejemplo, si se construye un sitio web como el que el equipo ya ha elaborado con éxito, con las mismas herramientas y los mismos requisitos— en cuyo caso, no tiene que considerarse en estas primeras iteraciones. En este caso, podrían implementarse características o casos de uso críticos, pero no significativos desde el punto de vista de la arquitectura.

Es en esta fase donde el libro se centra en una introducción al A/DOO, aplicando UML, patrones y arquitectura.

8.1. Punto de control: ¿qué sucedió en el inicio?

La etapa de inicio del proyecto de PDV NuevaEra puede durar sólo una semana. Los artefactos creados deberían ser breves e incompletos, la etapa rápida, y la investigación ligera.

No es la fase de requisitos del proyecto, sino una etapa breve para determinar la viabilidad, riesgo y alcance básicos, y decidir si merece la pena más investigación seria en el proyecto, que tendrá lugar en la elaboración. No se han cubierto todas las actividades que podrían ocurrir, de manera razonable, en la fase de inicio; este estudio resalta los artefactos orientados a los requisitos. Algunas de las actividades y artefactos posibles en la fase de inicio comprenden:

- Un breve taller de requisitos.
- La mayoría de los actores, objetivos y casos de usos, con los nombres.
- La mayoría de los casos de uso escritos en formato breve; del 10 al 20% de los casos de uso se escriben en detalle en formato completo para mejorar la comprensión del alcance y la complejidad.
- Identificación de la mayoría de los requisitos de calidad influyentes y de riesgo.
- Escritura de la primera versión de la Visión y Especificación Complementaria.
- Lista de riesgos.
 - Por ejemplo, el director en realidad quiere una demostración en la feria comercial POSWorld en Hamburgo, dentro de 18 meses. Pero el esfuerzo para el desarrollo de la demo no puede ser ni siquiera estimado a grandes rasgos hasta que no se haga un estudio más profundo.
- Prototipos de pruebas de conceptos técnicos y otros estudios para explorar la viabilidad técnica de los requisitos especiales (“¿Funcionan los Java Swing de manera adecuada en pantallas táctiles?”).
- Prototipos orientados a la interfaz de usuario para clarificar la visión de los requisitos funcionales.
- Recomendaciones sobre qué componentes comprar/construir/reutilizar, que se refinarán en la elaboración.
 - Por ejemplo, una recomendación de compra de un paquete de cálculo de impuestos.
- Arquitectura de alto nivel *candidata* y componentes propuestos.
 - No se trata de una descripción detallada de la arquitectura, y no se pretende que sea final o correcta. Más bien, se refiere a una breve especulación que se va a utilizar como punto de partida del estudio en la elaboración. Por ejemplo, “una aplicación Java del lado del cliente, sin un servidor de aplicaciones, Oracle para la base de datos,...”. En la elaboración, podría probarse que merece la pena, o descubrir que es una mala idea y rechazarla.
- Plan para la primera iteración.
- Lista de herramientas candidatas.

8.2. En la elaboración

La elaboración es la serie inicial de iteraciones durante las que el equipo lleva a cabo un estudio serio, implementa (programas y pruebas) el núcleo central de la arquitectura, aclara la mayoría de los requisitos, y aborda las cuestiones de alto riesgo. En el UP, el “riesgo” incluye valor del negocio. Por tanto, el trabajo inicial podría incluir la implementación de los escenarios que se consideran importantes, pero que no son especialmente arriesgados desde el punto de vista técnico.

La elaboración, a menudo, consta de entre dos y cuatro iteraciones; se recomienda que cada iteración dure entre dos y seis semanas, a menos que el tamaño del equipo sea muy grande. Se fija la duración de cada iteración, entendiendo que se fija la fecha de finalización; si es probable que el equipo no cumpla con la fecha, algunos requisitos se colocan en la lista de tareas futuras, de manera que la iteración pueda concluir a tiempo, con una versión estable y que se pueda probar.

La elaboración no se corresponde con una fase de diseño o una fase en la que se desarrollan completamente los modelos preparándolos para que se implementen en la etapa de construcción —que sería un ejemplo de superposición de las ideas del desarrollo en cascada sobre el desarrollo iterativo y el UP—.

Durante esta fase, uno no está creando prototipos desechables; sino que el código y el diseño son porciones del sistema final con calidad de producción. En algunas descripciones del UP, el término, que puede ser origen de malentendidos, “**prototipo de la arquitectura**” se utiliza para describir el sistema parcial. La intención no es que sea un prototipo en el sentido de un experimento desechable; en el UP, significa un subconjunto de producción del sistema final. Más comúnmente se denomina **arquitectura ejecutable o base de la arquitectura**.

La elaboración en una frase

Construir el núcleo central de la arquitectura, resolver los elementos de alto riesgo, definir la mayoría de los requisitos, y estimar la planificación y los recursos globales.

Algunas de las ideas claves y buenas prácticas, que se pondrán de manifiesto en la elaboración, incluyen:

- Llevar a cabo iteraciones breves, de duración fija, dirigidas por el riesgo.
- Comenzar a programar pronto.
- Diseñar, implementar y probar, de manera adaptable, las partes básicas y arriesgadas de la arquitectura.
- Probar desde el principio, a menudo y de manera realista.
- Adaptar en base a la retroalimentación procedente de las pruebas, usuarios y desarrolladores.
- Escribir la mayoría de los casos de uso y otros requisitos en detalle, a través de una serie de talleres, uno por cada iteración de la elaboración.

¿Qué es significativo desde el punto de vista de la arquitectura en la elaboración?

Las iteraciones iniciales construyen y prueban el núcleo central de la arquitectura. Para el proyecto de PDV NuevaEra —y de hecho, para la mayoría— esto incluirá:

- Empleo de diseño e implementación “ancho y superficial”; o, como lo ha llamado Grady Booch, “diseño en las fronteras”.
 - Es decir, identificación de los diferentes procesos, capas, paquetes, y subsistemas, y sus responsabilidades e interfaces de alto nivel. Implementarlos parcialmente con el objeto de conectarlos y clarificar las interfaces. Los módulos contendrían, en su mayor parte, código “*stubbed*” que permite probar las conexiones e interfaces.
- Refinamiento de las interfaces locales y remotas inter-módulos (incluyendo los detalles más sutiles acerca de los parámetros y valores de retorno).
 - Por ejemplo, la interfaz del objeto que actuará de intermediario (*wrapper*) en el acceso al sistema de contabilidad de terceras partes.
 - La primera versión de una interfaz rara vez es perfecta. La preocupación inicial por insistir en la prueba, “provocar fallos” y refinamiento de las interfaces, ayuda más tarde al trabajo en paralelo de varios equipos, confiando en interfaces estables.
- Integración de los componentes existentes.
 - Por ejemplo, un calculador de impuestos.
- Implementación de escenarios simplificados de caminos de éxito y fracaso que impulsan el diseño, la implementación y las pruebas, a través de muchos componentes importantes.
 - Por ejemplo, el escenario principal de éxito de *Procesar Venta*, utilizando el escenario de extensión para el pago a crédito.

Las pruebas en la fase de elaboración son importantes, para obtener retroalimentación, adaptar y probar que el núcleo es robusto. Las pruebas iniciales del proyecto NuevaEra incluirán:

- Pruebas de usabilidad de la interfaz de usuario para *Procesar Venta*.
- Pruebas de recuperación cuando falla el servicio remoto, como la autorización de crédito.
- Pruebas de carga alta a los servicios remotos, como la carga al sistema de cálculo de impuestos remoto.

8.3. Planificación de la siguiente iteración

La planificación y gestión del proyecto son importantes pero son temas amplios. En esta sección presentamos brevemente algunas ideas claves, y se proporcionará una introducción en el Capítulo 36.

Organice los requisitos y las iteraciones según el riesgo, grado de cobertura y naturaleza crítica.

- **Riesgo:** comprende tanto la complejidad técnica como otros factores, como incertidumbre del esfuerzo o facilidad de uso.
- **Cobertura:** implica que todas las partes importantes del sistema se tratan, al menos, en las primeras iteraciones —quizás una implementación “amplia y superficial” a través de muchos componentes—.
- **Naturaleza crítica:** se refiere a las funciones de alto valor para el negocio.

Estos criterios se utilizan para priorizar el trabajo a través de las iteraciones. Los casos de uso, o los escenarios de los casos de uso, se clasifican para la implementación —las primeras iteraciones implementan los escenarios clasificados de rango alto—. Además, algunos requisitos se enuncian como características de alto nivel no relacionadas con ningún caso de uso en particular, como el servicio de registro. Éstas también se clasifican.

La clasificación se realiza antes de la Iteración 1, pero después otra vez antes de la Iteración 2, etcétera, cuando nuevos requisitos y nuevas interpretaciones influyan en el orden. Es decir, el plan es adaptable, en lugar de fijado de un modo especulativo al comienzo del proyecto.

Normalmente, surgirá una agrupación difusa de requisitos basada en algunas técnicas de clasificación de colaboración de grupos pequeños. Por ejemplo:

Rango	Requisito (Caso de Uso o Característica)	Comentario
Alto	Procesar Venta Registro (Logging) ...	Puntuación alta en todos los criterios de clasificación. Extendido. Difícil de añadir más tarde. ...
Medio	Mantener Usuarios ...	Afecta al subdominio de seguridad ...
Bajo	...	...

Basándonos en esta clasificación, vemos que algunos escenarios claves del caso de uso *Procesar Venta*, significativos desde el punto de vista de la arquitectura, deberían abordarse en las primeras iteraciones. Esta lista no es exhaustiva; también se abordarán otros requisitos. Además, en cada iteración se trabajará en un caso de uso, implícito o explícito, *Poner en Marcha*, para satisfacer sus necesidades de inicialización.

En términos de los artefactos del UP, unos pocos comentarios sobre esta información de planificación:

- Los requisitos elegidos para la siguiente iteración se listan brevemente en un **Plan de Iteración**. No se trata de un plan de todas las iteraciones, sólo un plan de la siguiente.
- Si la breve descripción del Plan de Iteración es insuficiente, se podría escribir con más detalle una tarea o requisitos en una **Solicitud de Cambio**, y dársela al equipo responsable.
- La clasificación de requisitos se recoge en el **Plan de Desarrollo de Software**.

8.4. Requisitos y énfasis de la iteración 1: habilidades de A/DOO fundamentales

En este caso de estudio, la Iteración 1 de la fase de elaboración hace hincapié en una gama de habilidades de A/DOO fundamentales y comunes, utilizadas en la construcción de sistemas de objetos, como la asignación de las responsabilidades a los objetos. Por supuesto, se necesitan muchas otras habilidades y etapas —como el diseño de bases de datos, ingeniería de usabilidad y diseño de interfaces de usuario— para construir software, pero quedan fuera del alcance de esta introducción al A/DOO y al UP.

Requisitos de la iteración 1

Los requisitos para la primera iteración de la aplicación PDV NuevaEra son los siguientes:

- Implementación de un escenario clave básico del caso de uso *Procesar Venta*: entrada de artículos y recepción del pago en efectivo.
- Implementación de un caso de uso *Poner en Marcha*, necesario para dar soporte a las necesidades de inicialización de la iteración.
- No se maneja nada especial o complejo, sólo un simple escenario que acaba con éxito, y el diseño e implementación que lo soporte.
- No hay colaboración con servicios externos, como el sistema de cálculo de impuestos o base de datos de artículos.
- No se aplican reglas complejas para fijar los precios.

Aunque no se cubre, también se abordaría el diseño e implementación de las interfaces de usuario de soporte.

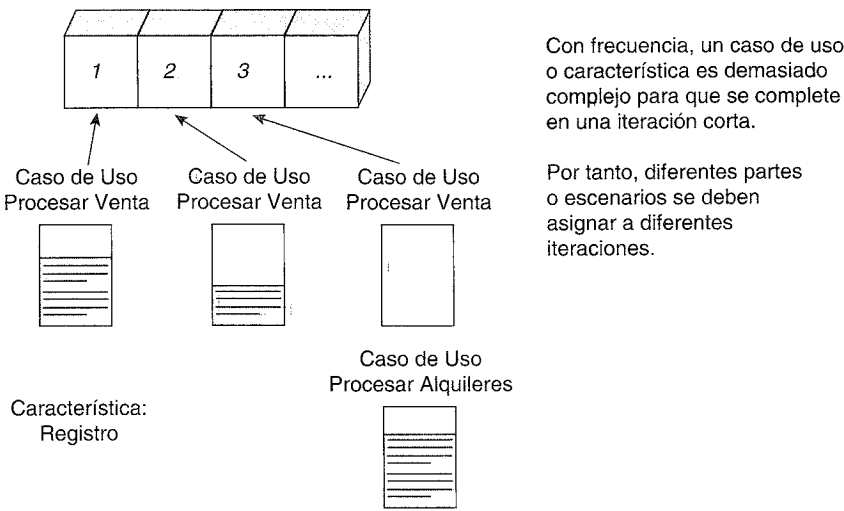
Las siguientes iteraciones extenderán esta base.

Desarrollo incremental para el mismo caso de uso a través de las iteraciones

Nótese que no todos los requisitos del caso de uso *Procesar Venta* se han tratado en la iteración 1. Es normal continuar con el trabajo de varios escenarios o características del mismo caso de uso en varias iteraciones y gradualmente ampliar el sistema para, al final, tratar toda la funcionalidad requerida (ver Figura 8.1). Por otro lado, los casos de uso simples y cortos, podrían completarse en una iteración.

8.5. ¿Qué artefactos podrían crearse en la elaboración?

La Tabla 8.1 lista artefactos de *muestra* que podrían crearse en la elaboración, e indica qué cuestión abordan. Los capítulos siguientes presentarán algunos de ellos con mayor



Con frecuencia, un caso de uso o característica es demasiado complejo para que se complete en una iteración corta. Por tanto, diferentes partes o escenarios se deben asignar a diferentes iteraciones.

Figura 8.1. La implementación del caso de uso puede extenderse a lo largo de varias iteraciones.

Tabla 8.1. Muestra de Artefactos de elaboración, excluyendo aquellos que se crearon en la fase de inicio.

Artefacto	Comentario
Modelo del Dominio	Es una visualización de los conceptos del dominio; es similar al modelo de información estático de las entidades del dominio.
Modelo de Diseño	Es el conjunto de diagramas que describen el diseño lógico. Comprende los diagramas de clases software, diagramas de interacción, diagramas de paquetes, etcétera.
Documento de la Arquitectura Software	Una ayuda de aprendizaje que resume las cuestiones claves de la arquitectura y cómo se resuelven en el diseño. Es un resumen de las ideas destacadas del diseño y su motivación en el sistema.
Modelo de Datos	Incluye los esquemas de bases de datos, y las estrategias de transformación entre representaciones de objetos y no objetuales.
Modelo de Pruebas	Una descripción de lo que se probará y cómo.
Modelo de Implementación	Se corresponde con la implementación real —el código fuente, ejecutables, base de datos, etcétera—.
Guiones de Caso de Uso, Prototipos UI	Descripción de la interfaz de usuario, caminos de navegación, modelos de facilidad de uso, etcétera.

detalle, en especial el Modelo del Dominio y el Modelo de Diseño. Por brevedad, la tabla excluye los artefactos que pueden crearse en la fase de inicio (y se listaron en el Capítulo 4); presenta los artefactos que es más probable que se creen en la elaboración. Nótese que no se completarán en una iteración; sino que se refinarán a lo largo de una serie de iteraciones.

8.6. No se entendió la elaboración cuando...

- La duración es superior a “unos pocos” meses en la mayoría de los proyectos.
- Sólo comprende una iteración (con raras excepciones para los problemas bien entendidos).
- La mayoría de los requisitos se definieron antes de la elaboración.
- Los elementos arriesgados y el núcleo de la arquitectura no se abordan.
- El resultado no es una arquitectura *ejecutable*; no hay programación de código de producción.
- Se considera fundamentalmente una fase de requisitos, que precede a una fase de implementación en la construcción.
- Se intenta llevar a cabo un diseño completo y cuidadoso antes de la programación.
- Existe una retroalimentación y adaptación mínima; los usuarios no se involucran continuamente en la evaluación y retroalimentación.
- No se llevan a cabo pruebas realistas en las primeras etapas.
- La arquitectura se termina de forma especulativa antes de la programación.
- Se considera una etapa para realizar la programación de pruebas de conceptos, en lugar de programar la arquitectura ejecutable básica de producción.
- No se realizan varios talleres de requisitos breves que adaptan y refinan los requisitos en base a la retroalimentación de las iteraciones anteriores y la actual.

Si un proyecto presenta estos síntomas, no se ha entendido la fase de elaboración.

ELABORACIÓN EN LA ITERACIÓN 1

MODELO DE CASOS DE USO: REPRESENTACIÓN DE LOS DIAGRAMAS DE SECUENCIA DEL SISTEMA

En teoría, no hay diferencia entre teoría y práctica. Pero, en la práctica, la hay.

Jan L. A. van de Snepscheut

Objetivos

- Identificar los eventos de sistema.
 - Crear diagramas de secuencia del sistema para los casos de uso.
-

Sigamos con la iteración 1

El proyecto PDV NuevaEra ha entrado en la primera iteración de desarrollo real. En la fase de inicio se ha realizado un trabajo ligero de requisitos para ayudar a decidir si merecía la pena más investigación seria en el proyecto. Se ha completado la planificación de la primera iteración, y se ha decidido abordar un escenario sencillo de éxito del caso de uso *Procesar Venta* para, únicamente, el pago en efectivo (sin colaboraciones remotas), con el objetivo de comenzar un diseño e implementación “amplio y superficial”, que incluya muchos de los elementos importantes del nuevo sistema. En la primera iteración, tienen lugar muchas tareas relacionadas con el establecimiento del entorno (herramientas, personal, procesos e instalación) que no serán consideradas.

Más bien, volvemos nuestra atención a los casos de usos y al análisis del modelado del dominio. Antes de empezar el trabajo de diseño de la iteración 1, resultará útil realizar un estudio adicional del dominio del problema. Parte de este estudio comprende la

aclaración de los eventos del sistema de entrada y salida relacionados con nuestro sistema, que puede representarse en diagramas de secuencia UML.

Introducción

Un diagrama de secuencia del sistema es un artefacto creado de manera rápida y fácil, que muestra los eventos de entrada y salida relacionados con el sistema que se está estudiando. UML incluye la notación de los diagramas de secuencia para representar los eventos que parten de los actores externos hacia el sistema.

9.1. Comportamiento del sistema

Antes de continuar con el diseño lógico de cómo funcionará la aplicación software, es conveniente estudiar y definir su comportamiento como una “caja negra”. El **comportamiento del sistema** es una descripción de *qué* hace el sistema, sin explicar cómo lo hace. Una parte de esa descripción es un diagrama de secuencia del sistema. Otras partes comprenden los casos de uso y los contratos del sistema (que se presentarán después).

9.2. Diagramas de secuencia del sistema

Los casos de uso describen cómo interactúan los actores externos con el sistema software que estamos interesados en crear. Durante esta interacción, un actor genera eventos sobre un sistema, normalmente solicitando alguna operación como respuesta. Por ejemplo, cuando un cajero inserta el ID de un artículo está solicitando al sistema PDV que registre la venta de ese artículo. Ese evento de solicitud inicia una operación sobre el sistema.

Es deseable aislar e ilustrar las operaciones que un actor externo solicita a un sistema, porque constituyen una parte importante de la comprensión del comportamiento del sistema. UML incluye los **diagramas de secuencia** como notación que puede representar las interacciones de los actores y las operaciones que inician.

Un **diagrama de secuencia del sistema** (DSS) es un dibujo que muestra, para un escenario específico de un caso de uso, los eventos que generan los actores externos, el orden y los eventos entre los sistemas. Todos los sistemas se tratan como cajas negras; los diagramas destacan los eventos que cruzan los límites del sistema desde los actores a los sistemas.

Debería hacerse un DSS para el escenario principal de éxito del caso de uso, y los escenarios alternativos complejos o frecuentes.

UML no define nada denominado diagrama de secuencia “del sistema”, sino simplemente diagrama de secuencia. La calificación se utiliza para subrayar su aplicación para representar sistemas como cajas negras. Posteriormente, se utilizarán los diagramas de secuencia en otro contexto —para ilustrar el diseño de la interacción entre objetos software para completar un trabajo.

9.3. Ejemplo de un DSS

Un DSS muestra, para un curso de eventos específico en un caso de uso, los actores externos que interaccionan directamente con el sistema, el sistema (como una caja negra) y los eventos del sistema que genera el actor (ver Figura 9.1). El tiempo avanza hacia abajo, y la ordenación de los eventos debería seguir su orden en el caso de uso.

Los eventos del sistema podrían contener parámetros.

Este ejemplo muestra el escenario principal de éxito del caso de uso *Procesar Venta*. Se indica que el cajero genera los eventos del sistema *crearNuevaVenta*, *introducirArticulo*, *finalizarVenta*, y *realizarPago*.

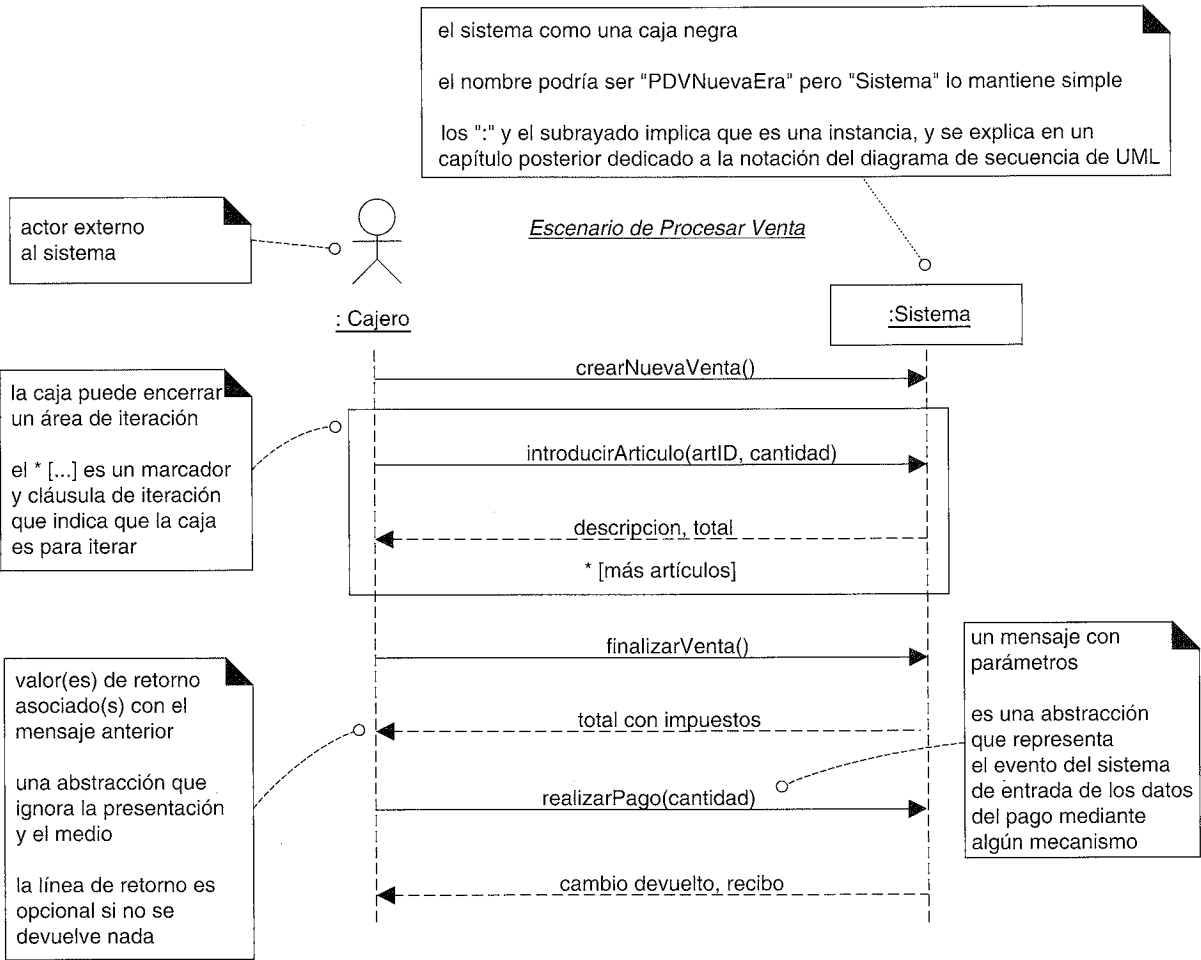


Figura 9.1. DSS para un escenario de *Procesar Venta*.

9.4. DSS entre sistemas

Los DSS también pueden utilizarse para ilustrar las colaboraciones entre sistemas, como entre el PDV NuevaEra y el sistema externo que autoriza pagos a crédito. Sin em-

bargo, se pospone hasta una iteración posterior en el caso de estudio, puesto que esta iteración no incluye las colaboraciones con sistemas remotos.

9.5. DSS y los casos de uso

Un DSS muestra los eventos del sistema para un escenario de un caso de uso, por tanto, se genera para el estudio de un caso de uso (ver Figura 9.2).

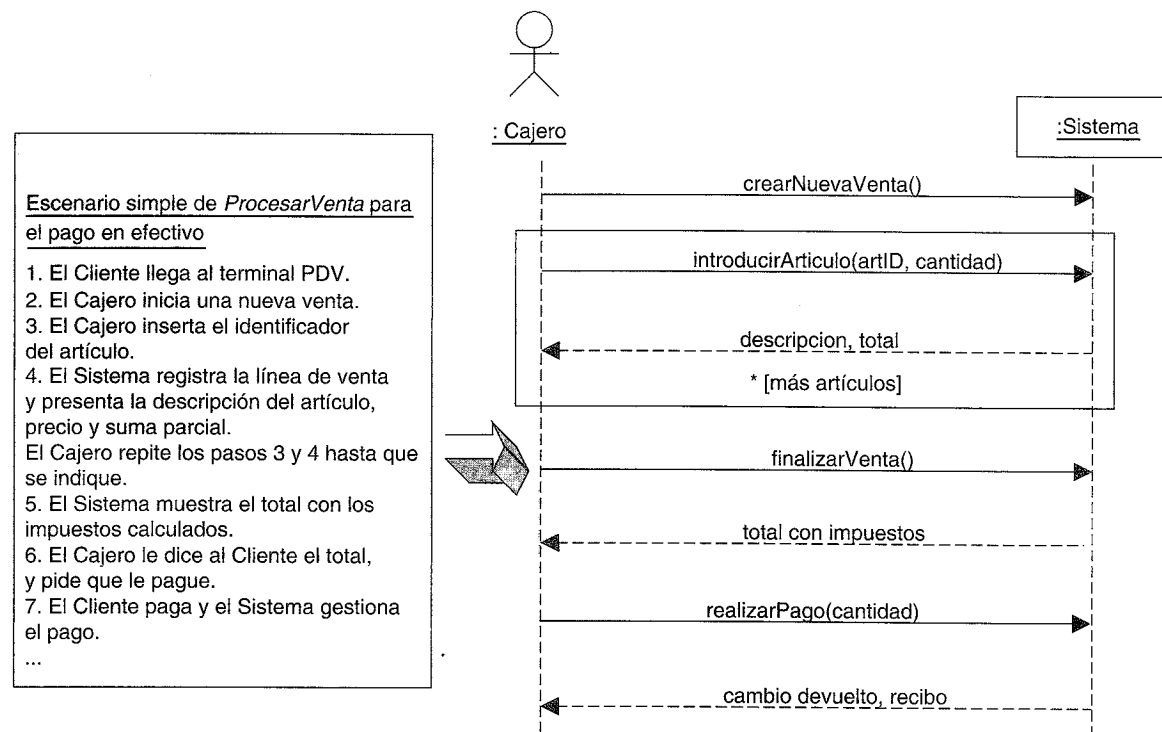


Figura 9.2. Los DSS se derivan de los casos de uso.

9.6. Eventos del sistema y los límites del sistema

Para identificar los eventos del sistema, es necesario tener claros los límites del sistema, como se presentó en el capítulo anterior sobre los casos de uso. Por lo que toca al desarrollo de software, el límite del sistema normalmente se elige para que sea el propio sistema software (y posiblemente hardware); en este contexto, un evento del sistema es un evento externo que lanza un estímulo directamente al software (ver Figura 9.3).

Consideremos el caso de uso *Procesar Venta* para identificar los eventos del sistema. Primero, debemos determinar los actores que interactúan directamente con el sistema software. El cliente interactúa con el cajero, pero para este escenario simple de pago en efectivo, no interactúa directamente con el sistema PDV —sólo lo hace el cajero—. Por tanto, el cliente no es un generador de eventos del sistema; sólo lo es el cajero.

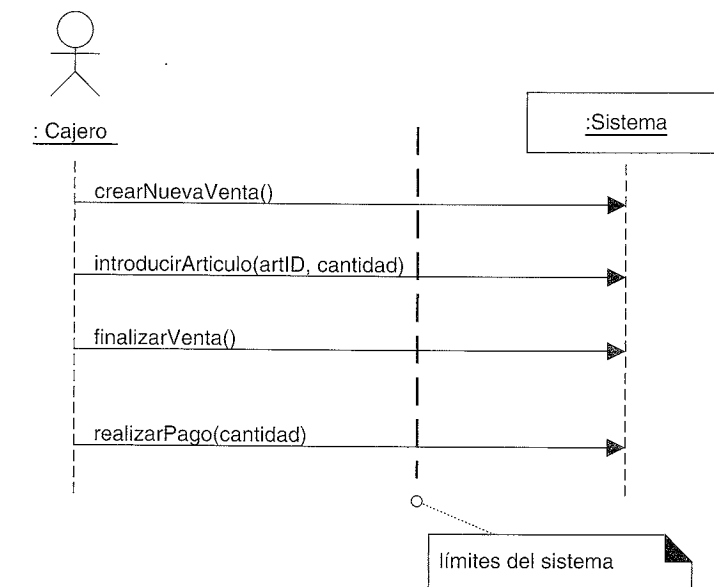


Figura 9.3. Definición de los límites del sistema.

9.7. Asignación de nombres a los eventos y operaciones

Los eventos del sistema (y sus operaciones del sistema asociadas) deberían expresarse al nivel de intenciones en lugar de en términos del medio de entrada físico o a nivel de elementos de la interfaz de usuario.

También se mejora la claridad, el comenzar el nombre de un evento del sistema con un verbo (añadir..., insertar..., finalizar..., crear...), como en la Figura 9.4, puesto que resalta la orientación de orden de estos eventos.

Así, “introducirArticulo” es mejor que “escanear” (esto es, escanear con láser) porque captura la intención de la operación, al mismo tiempo que permanece abstracta y sin compromiso respecto a las elecciones de diseño sobre qué interfaz utilizar para capturar el evento del sistema.

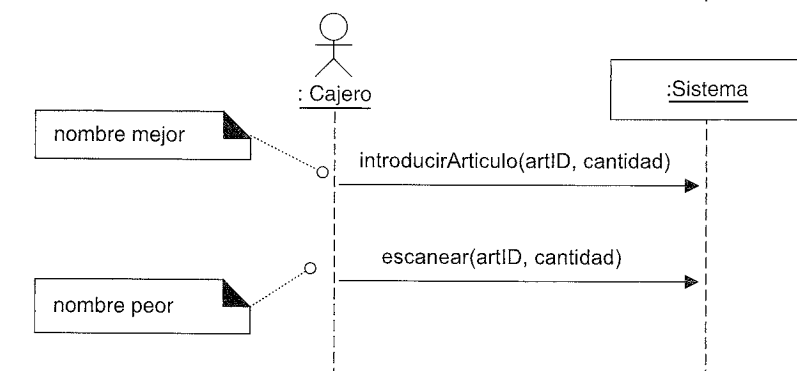


Figura 9.4. Elección de los nombres de los eventos y las operaciones a un nivel abstracto.

9.8. Mostrar el texto del caso de uso

A veces, es deseable mostrar al menos fragmentos del texto del caso de uso del escenario, con el fin de aclarar o enriquecer las dos vistas (ver Figura 9.5). El texto proporciona los detalles y el contexto; el diagrama resume visualmente la interacción.

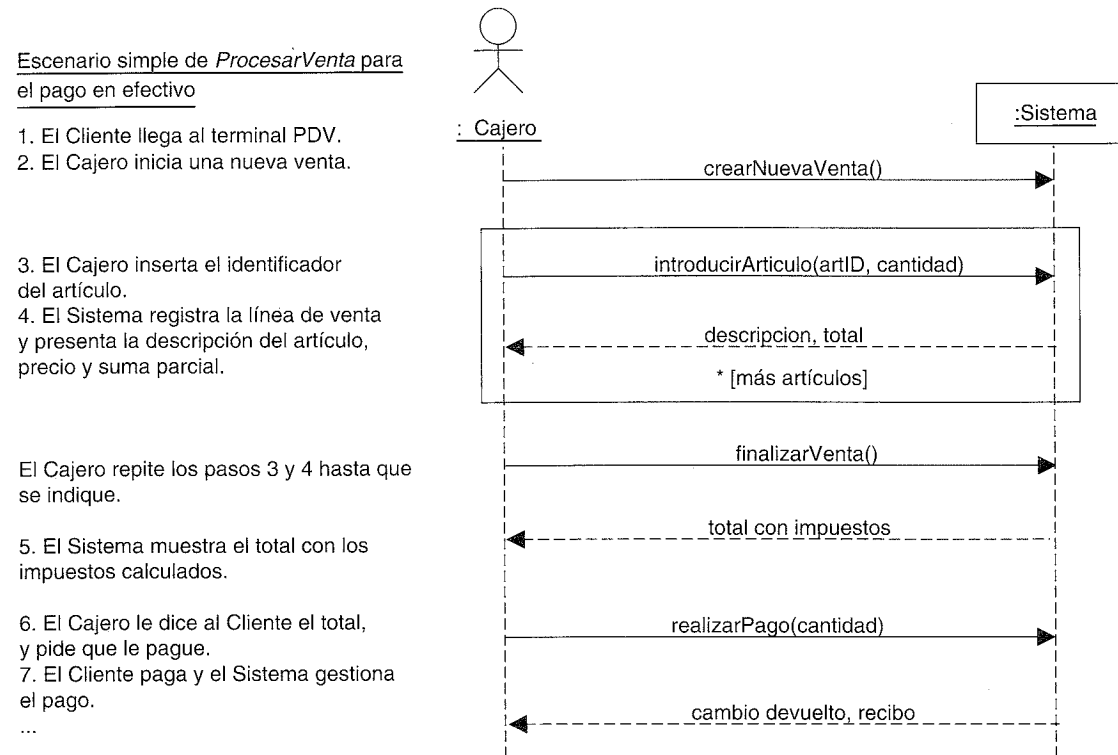


Figura 9.5. DSS con texto del caso de uso.

9.9. Los DSS y el Glosario

Los términos representados en los DSS (operaciones, parámetros, valores de retorno) son concisos. Éstos podrían necesitar una explicación más adecuada de manera que, durante el trabajo de diseño, esté claro qué es lo que entra y lo que sale. Si no se explicó en los casos de uso, podría utilizarse el Glosario.

Sin embargo, como siempre que discutimos la creación de artefactos distintos al código (lo esencial del proyecto), hay que ser desconfiado. Con los datos del Glosario debería hacerse algún uso o decisión realmente significativa, de lo contrario es un trabajo innecesario de poco valor.

9.10. DSS en el UP

Los DSSs forman parte del Modelo de Casos de Uso —una visualización de las interacciones implicadas en los casos de uso—. Los DSSs no se mencionaron explícitamente en la

descripción original del UP, aunque los creadores del UP eran conscientes y entendían la utilidad de tales diagramas. Los DSS son un ejemplo de los muchos posibles artefactos o actividades de análisis y diseño de utilidad que los documentos UP o el RUP no mencionan.

Fases

Inicio: Los DSS no se incentivan normalmente en la fase de inicio.

Elaboración: La mayoría de los DSS se crean durante la elaboración, cuando es útil identificar los detalles de los eventos del sistema para poner en claro cuáles son las operaciones que se deben diseñar que gestione el sistema, escribir los contratos de las operaciones del sistema (se presentarán en el Capítulo 13), y posiblemente, dar soporte a la estimación (por ejemplo, macroestimación con puntos de función no ajustados y COCOMO II).

Nótese que no es necesario crear DSS para todos los escenarios de todos los casos de uso —al menos no a la vez. Sino que, se crearán sólo para algunos escenarios seleccionados de la iteración actual.

Finalmente, sólo debería llevar unos pocos minutos o una media hora la creación de los DSSs.

Tabla 9.1. Muestra de los artefactos UP y evolución temporal. c – comenzar; r – refinar.

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso (DSS)	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

9.11. Lecturas adicionales

Durante décadas, se ha utilizado ampliamente variaciones de los diagramas que ilustran los eventos de entrada/salida de un sistema considerado como una caja negra; por ejemplo, en telecomunicaciones como diagramas de flujo de llamadas. Especialmente, se hicieron populares en los métodos orientados a objetos a través de su uso en el método Fusion [Coleman+94], que proporciona un ejemplo detallado de la relación de los DSSs y las operaciones del sistema con otros artefactos del análisis y diseño.

9.12. Artefactos del UP

La Figura 9.6 presenta ejemplos de relaciones de los DSSs con otros artefactos.

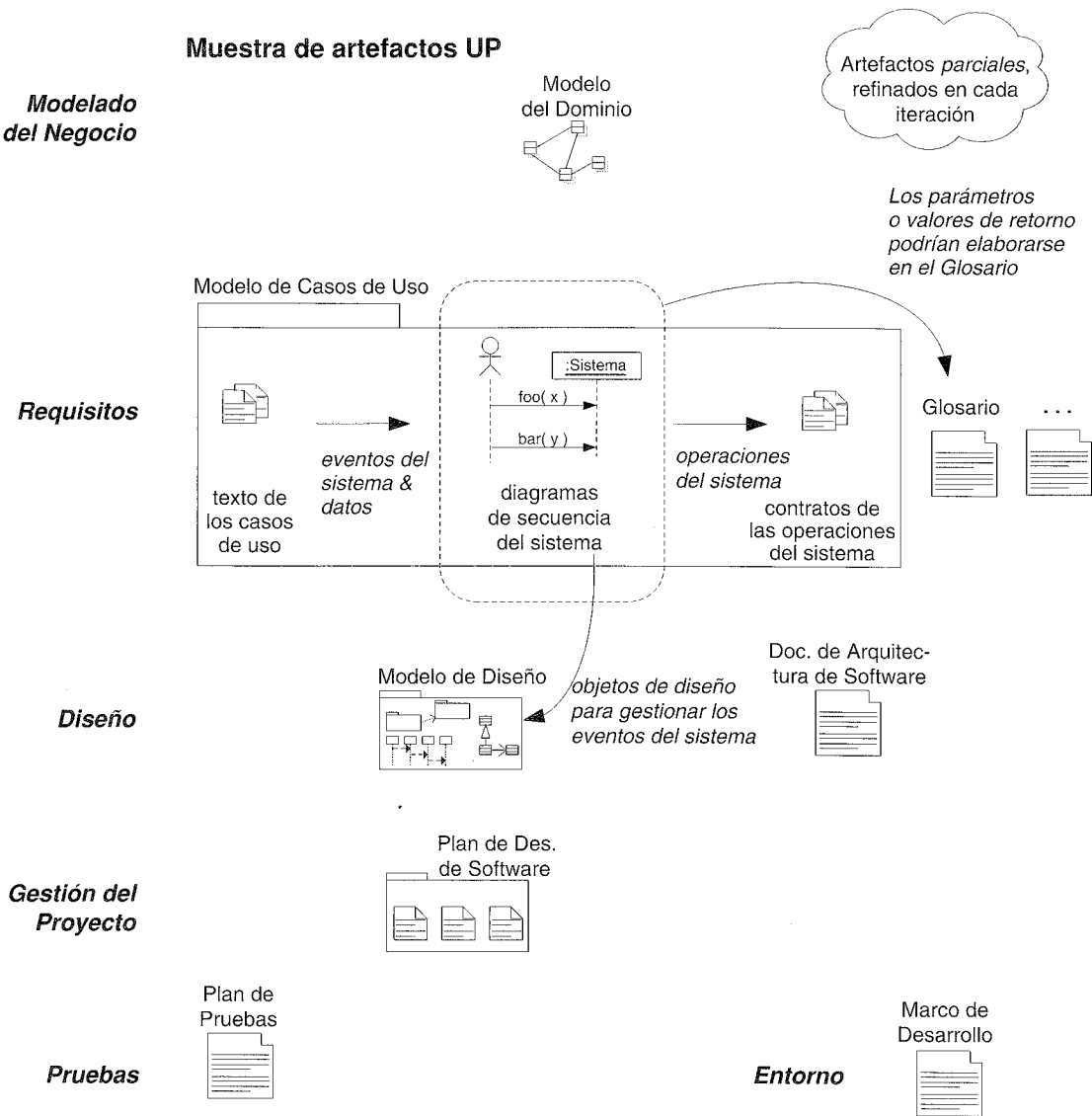


Figura 9.6. Muestra de la influencia de los artefactos UP.

MODELO DEL DOMINIO: VISUALIZACIÓN DE CONCEPTOS

Todo está muy bien en la práctica, pero nunca funcionará en la teoría.

Máxima anónima sobre la gestión

Objetivos

- Identificar las clases conceptuales relacionadas con los requisitos de la iteración actual.
- Crear un modelo del dominio inicial.
- Distinguir entre atributos correctos e incorrectos.
- Añadir las clases conceptuales de *especificación*, cuando sea oportuno.
- Comparar y contrastar las vistas conceptual y de implementación.

Introducción

Un modelo del dominio se utiliza con frecuencia como fuente de inspiración para el diseño de los objetos software, y será una entrada necesaria para varios de los siguientes artefactos que se presentan en este libro. Por tanto, es importante leer este capítulo si el lector no está familiarizado con el tema del modelado del dominio.

Un modelo del dominio muestra (a los modeladores) clases conceptuales significativas en un dominio del problema; es el artefacto más importante que se crea durante el análisis orientado a objetos¹. Este capítulo estudia técnicas introductorias a la creación de modelos del dominio. Los siguientes dos capítulos tratarán más extensamente las técnicas de modelado del dominio —añadiendo atributos y asociaciones—.

¹ Los casos de uso son un importante artefacto del análisis de requisitos, pero no son orientados a *objetos*. Ponen de relieve una vista de procesos del dominio.

La identificación de un conjunto rico de objetos o clases conceptuales es una parte esencial del análisis orientado a objetos, y bien merece la pena el esfuerzo en relación con los beneficios durante el trabajo de diseño e implementación.

La identificación de las clases conceptuales forma parte del estudio del dominio del problema. UML contiene notación, en forma de diagramas de clases, para representar los modelos del dominio.

Idea clave

Un modelo del dominio es una representación de las clases conceptuales del mundo real, no de componentes software. No se trata de un conjunto de diagramas que describen clases software, u objetos software con responsabilidades.

10.1. Modelos del dominio

La etapa orientada a *objetos* esencial del análisis o investigación es la descomposición de un dominio de interés en clases conceptuales individuales u objetos —las cosas de las que somos conscientes—. Un **modelo del dominio** es una representación *visual* de las clases conceptuales u objetos del mundo real en un dominio de interés [MO95, Fowler96]. También se les denomina **modelos conceptuales** (término utilizado en la primera edición de este libro), **modelo de objetos del dominio** y **modelos de objetos de análisis**².

El UP define un Modelo de Dominio³ como uno de los artefactos que podrían crearse en la disciplina del Modelado del Negocio.

Utilizando la notación UML, un modelo del dominio se representa con un conjunto de **diagramas de clases** en los que no se define ninguna operación. Pueden mostrar:

- Objetos del dominio o clases conceptuales.
- Asociaciones entre las clases conceptuales.
- Atributos de las clases conceptuales.

Por ejemplo, la Figura 10.1 muestra un modelo del dominio parcial. Ilustra que las clases conceptuales *Pago* y *Venta* son significativas en este dominio, que un *Pago* está relacionado con una *Venta* de un modo que es significativo hacer notar, y que una *Venta* tiene una fecha y una hora. Los detalles de la notación no son importantes en este momento.

Idea clave: Modelo del dominio—un diccionario visual de abstracciones

Por favor, reflexione un momento sobre la Figura 10.1. La figura visualiza y relaciona algunas palabras o clases conceptuales del dominio. También describe una *abstracción* de

² También están relacionados con los modelos conceptuales entidad-relación, que son capaces de mostrar vistas de los dominios puramente conceptuales, pero que han sido ampliamente re-interpretados como modelos de datos para el diseño de bases de datos. Los modelos del dominio no son modelos de datos.

³ Se utiliza Modelo del Dominio en mayúsculas cuando deseo resaltar que es un modelo oficial definido en el UP, frente al concepto bien conocido de “modelos de dominio”.

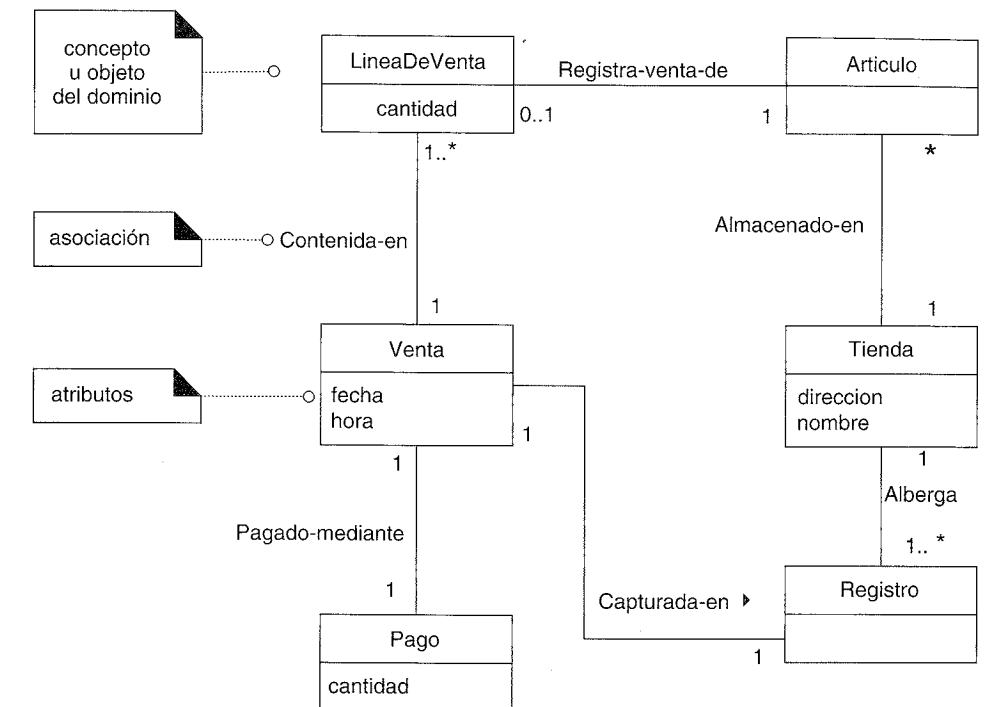


Figura 10.1. Modelo del dominio parcial —un diccionario visual—. El número de cada extremo de la línea indica la multiplicidad, que se describirá en un capítulo posterior.

las clases conceptuales, porque hay muchas cosas que uno podría comunicar sobre los registros, las ventas, etcétera. El modelo muestra una vista parcial, o abstracción, e ignora detalles sin interés (para el modelador).

La información que presenta (utilizando la notación UML) podría, de manera alternativa, haberse expresado en prosa, mediante sentencias en el Glosario o en algún otro sitio. Pero es fácil entender los distintos elementos y sus relaciones mediante este lenguaje visual, puesto que un porcentaje significativo del cerebro toma parte en el procesamiento visual —es una cualidad de los humanos—.

Por tanto, el modelo del dominio podría considerarse como un *diccionario visual* de las abstracciones relevantes, vocabulario del dominio e información del dominio.

Los modelos del dominio no son modelos de componentes software

Un modelo del dominio, como se muestra en la Figura 10.2, es una representación de las cosas del mundo real del dominio de interés, *no* de componentes software, como una clase Java o C++ (ver Figura 10.3), u objetos software con responsabilidades. Por tanto, los siguientes elementos no son adecuados en un modelo del dominio:

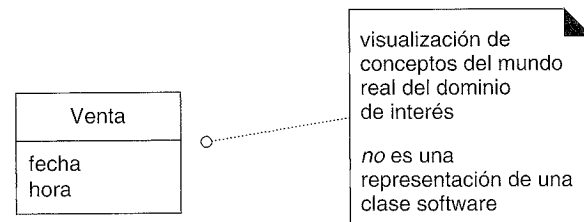


Figura 10.2. Un modelo del dominio muestra clases conceptuales del mundo real, no clases software.

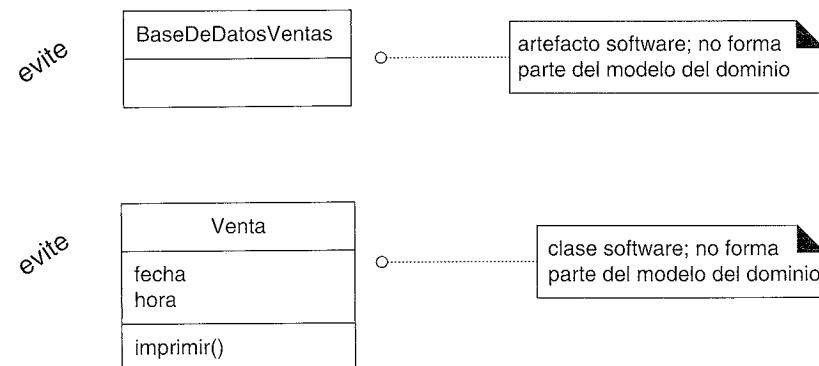


Figura 10.3. Un modelo del dominio no muestra artefactos software o clases.

- Artefactos software, como una ventana o una base de datos, a menos que el dominio que se esté modelando sea de conceptos software, como un modelo de interfaces de usuario gráficas.
- Responsabilidades o métodos⁴.

Clases conceptuales

El modelo del dominio muestra las clases conceptuales o vocabulario del dominio. Informalmente, una clase conceptual es una idea, cosa u objeto. Más formalmente, una clase conceptual podría considerarse en términos de su símbolo, intensión, y extensión [MO95] (ver Figura 10.4).

- **Símbolo:** palabras o imágenes que representan una clase conceptual.
- **Intensión:** la definición de una clase conceptual.
- **Extensión:** el conjunto de ejemplos a los que se aplica la clase conceptual.

Por ejemplo, considere la clase conceptual para el evento de una transacción de compra. Podría elegir nombrarla con el símbolo *Venta*. La intensión de una *Venta* podría es-

⁴ En el modelado de objetos, normalmente hablamos de responsabilidades relacionadas con los componentes software. Y los métodos son puramente conceptos software. Pero el modelo de dominio describe conceptos del mundo real, no componentes software. Es importante considerar las responsabilidades de los objetos durante el trabajo de *diseño*; sólo que no forma parte de este modelo. Un caso válido en el que se podrían mostrar las responsabilidades en un modelo de dominio es si incluye roles de trabajadores humanos (como el Cajero), y el modelador desea recoger las responsabilidades de estos trabajadores humanos.

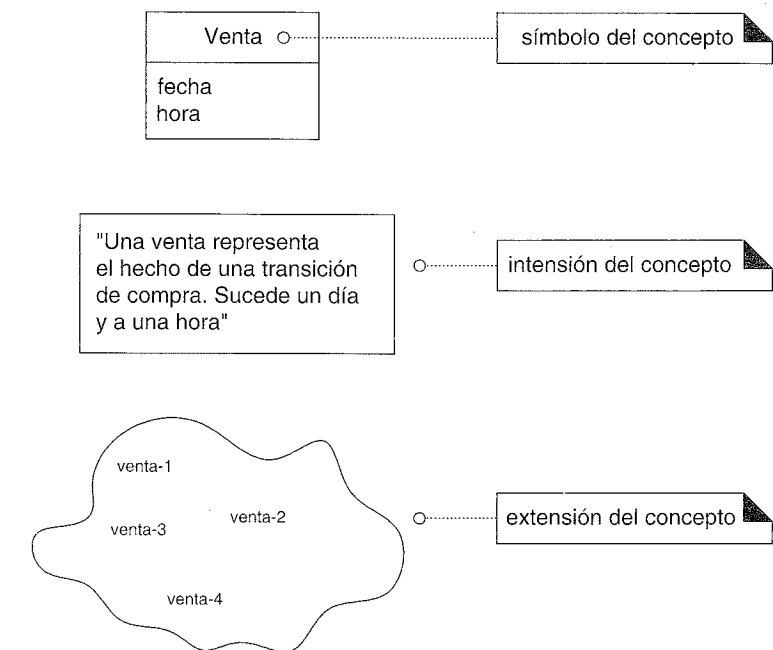


Figura 10.4. Una clase conceptual tiene un símbolo, una intensión y una extensión.

tablecer que “representa el hecho de una transacción de una compra, y tiene una fecha y una hora”. La extensión de una *Venta* la forman todos los ejemplos de ventas; en otras palabras, el conjunto de todas las ventas.

Cuando creamos un modelo del dominio, normalmente, el símbolo y la vista intensional de la clase conceptual son los que tienen un mayor interés práctico.

Modelos y descomposición del dominio

Los problemas del software pueden ser complejos; la descomposición —divide y vencerás— es una estrategia común para tratar esta complejidad mediante la división del espacio del problema en unidades fáciles de comprender. En el **análisis estructurado**, la dimensión de la descomposición es por procesos o por *funciones*. Sin embargo, en el análisis orientado a objetos, la dimensión de la descomposición es fundamentalmente por cosas o entidades del dominio.

Una diferencia esencial entre el análisis orientado a objetos y el estructurado es: la división por clases conceptuales (objetos) en lugar de la división por funciones.

Por tanto, la principal tarea del análisis es identificar diferentes conceptos en el dominio del problema y documentar el resultado en un modelo del dominio.

Clases conceptuales en el dominio de ventas

Por ejemplo, en el dominio de ventas en una tienda del mundo real, existen las clases conceptuales de *Tienda*, *Registro* y *Venta*. Por tanto, nuestro modelo del dominio, mostrado en la Figura 10.5, podría incluir *Tienda*, *Registro* y *Venta*.



Figura 10.5. Modelo del dominio parcial en el dominio de la tienda.

10.2. Identificación de las clases conceptuales

Nuestro objetivo es crear un modelo del dominio de clases conceptuales interesantes o significativas del dominio de interés (ventas). En este caso, eso significa conceptos relacionados con el caso de uso *Procesar Venta*.

En el desarrollo iterativo, uno incrementalmente construye un modelo del dominio a lo largo de varias iteraciones en la fase de elaboración. En cada una, el modelo de dominio se limita a los escenarios anteriores y actual en estudio, en lugar de un modelo de “gran explosión”, que en las primeras etapas intenta capturar todas las posibles clases conceptuales y las relaciones. Por ejemplo, esta iteración está limitada al escenario de pago en efectivo de *Procesar Venta*; por tanto, se creará un modelo de dominio parcial únicamente para reflejar eso —no más—.

La tarea central es, por tanto, identificar las clases conceptuales relacionadas con el escenario que se está diseñando.

A continuación presentamos una guía útil para la identificación de las clases conceptuales:

Es mejor especificar en exceso un modelo del dominio con muchas clases conceptuales de grano fino que especificar por defecto.

No piense que un modelo de dominio es mejor si contiene pocas clases conceptuales; suele ser verdad justamente lo contrario.

Es normal obviar clases conceptuales durante la etapa de identificación inicial, y descubrirlas más tarde al considerar los atributos y asociaciones, o durante el trabajo de diseño. Cuando se encuentren, se pueden añadir al modelo del dominio.

No excluya una clase conceptual simplemente porque los requisitos no indican ninguna necesidad obvia para registrar información sobre ella (un criterio común en el modelado de datos para el diseño de bases de datos relacionales, pero no relevante en el modelado del dominio), o porque la clase conceptual no tiene atributos.

Es válido tener clases conceptuales sin atributos, o clases conceptuales con un rol puramente de comportamiento en el dominio, en lugar de un rol de información.

Estrategias para identificar clases conceptuales

En las siguientes secciones se presentan dos técnicas:

- 1. Utilización de una lista de categorías de clases conceptuales.
- 2. Identificación de frases nominales.

Otra excelente técnica para el modelado del dominio es el uso de **patrones de análisis**, que son modelos de dominios parciales existentes creados por expertos, utilizando libros publicados como *Analysis Patterns* [Fowler96] y *Data Model Patterns* [Hay96].

Utilización de una lista de categorías de clases conceptuales

Comience la creación de un modelo del dominio haciendo una lista de clases conceptuales candidatas. La Tabla 10.1 contiene muchas categorías habituales que, normalmente, merece la pena tener en cuenta, aunque no en ningún orden particular de importancia. Los ejemplos se han extraído del dominio de las tiendas y las reservas de vuelos.

Tabla 10.1. Lista de categorías de clases conceptuales.

Categoría de clase conceptual	Ejemplos
objetos tangibles o físicos	Registro Avion
especificaciones, diseños o descripciones de la cosas	EspecificacionDelProducto DescripcionDelVuelo
lugares	Tienda
transacciones	Venta, Pago Reserva
líneas de la transacción	LineaDeVenta
roles de la gente	Cajero Piloto
contenedores de otras cosas	Tienda, Lata Avion
cosas en un contenedor	Articulo Pasajero
otros sistemas informáticos o electromecánicos externos al sistema	SistemaAutorizacionPagoCredito ControlDeTraficoAereo
conceptos abstractos	Ansia Acrofobia
organizaciones	DepartamentoDeVentas CompañiaAerea
hechos	Venta, Pago, Reunion Vuelo, Colisión, Aterrizaje

continúa

Tabla 10.1. Lista de categorías de clases conceptuales. (Continuación)

Categoría de clase conceptual	Ejemplos
procesos (normalmente <i>no</i> se representan como conceptos, pero podría ocurrir)	<i>VentaDeUnProducto</i> <i>ReservaUnAsiento</i>
reglas y políticas	<i>PoliticaDeReintegro</i> <i>PoliticaDeCancelación</i>
catálogos	<i>CatalogoDeProductos</i> <i>CatalogoDePiezas</i>
registros de finanzas, trabajo, contratos, cuestiones legales	<i>Recibo</i> , <i>LibroMayor</i> , <i>ContratoEmpleo</i> <i>RegistroMantenimiento</i>
instrumentos y servicios financieros	<i>LineaDeCredito</i> <i>Stock</i>
manuales, documentos, artículos de referencia, libros	<i>ListaDeCambiosDePreciosDiarios</i> <i>ManualReparaciones</i>

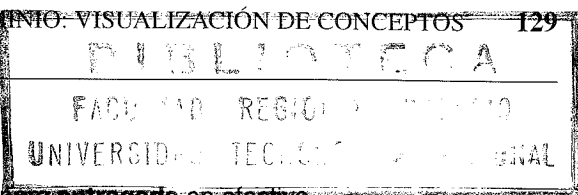
Descubrimiento de clases conceptuales mediante la identificación de frases nominales

Otra técnica útil (debido a su simplicidad) recomendada en [Abbot83] es el análisis lingüístico: identificar los nombres y frases nominales en las descripciones textuales de un dominio, y considerarlos como clases conceptuales o atributos candidatos.

Se debe tener cuidado con este método; no es posible realizar una correspondencia mecánica de nombres a clases, y las palabras en lenguaje natural son ambiguas.

En cualquier caso, es otra fuente de inspiración. Los casos de uso en formato completo constituyen una descripción excelente a partir de la cual extraer este análisis. Por ejemplo, se puede utilizar el escenario actual del caso de uso *Procesar Venta*.

- Escenario principal de éxito (o Flujo Básico):**
- 1. El **Ciente** llega a un **terminal PDV** con **mercancías** y/o **servicios** que comprar.
 - 2. El **Cajero** comienza una nueva **venta**.
 - 3. El **Cajero** introduce el **identificador del artículo**.
 - 4. El Sistema registra la **línea de la venta** y presenta la **descripción del artículo, precio** y **suma** parcial. El precio se calcula a partir de un conjunto de reglas de precios. El Cajero repite los pasos 3-4 hasta que se indique.
 - 5. El Sistema presenta el total con los **impuestos** calculados.
 - 6. El Cajero le dice al Cliente el total y solicita el **pago**.
 - 7. El Cliente paga y el Sistema gestiona el pago.
 - 8. El Sistema registra la **venta** completa y envía la información de la venta y el pago al sistema de **Contabilidad** externo (para la contabilidad y las **comisiones**) y al sistema de **Inventario** (para actualizar el inventario).
 - 9. El Sistema presenta el **recibo**.
 - 10. El Cliente se va con el recibo y las mercancías (si es el caso).



Extensiones (o Flujos Alternativos):

- ...
- 7a. Pago en efectivo:
- 1. El Cajero introduce la **cantidad** de dinero **entregada en efectivo**.
 - 2. El Sistema muestra la **cantidad** de dinero **a devolver** y abre el **cajón de caja**.
 - 3. El Cajero deposita el dinero entregado y devuelve el cambio al Cliente.
 - 4. El Sistema registra el pago en efectivo.

El modelo del dominio es una visualización de los conceptos del dominio y vocabulario relevantes. ¿Dónde se encuentran estos términos? En los casos de uso. Por tanto, constituyen una fuente rica a explorar mediante la identificación de frases nominales.

Algunas de las frases nominales son clases conceptuales candidatas, algunas podrían hacer referencia a clases conceptuales que se ignoran en esta iteración (por ejemplo, “Contabilidad” y “comisiones”), y algunas podrían ser atributos de las clases conceptuales. Por favor, diríjase a la siguiente sección y al capítulo sobre los atributos, para obtener los consejos que permiten diferenciar entre los dos.

Un punto débil de este enfoque es la imprecisión del lenguaje natural; frases nominales diferentes podrían representar la misma clase conceptual o atributo, entre otras ambigüedades. Sin embargo, se recomienda que se combine con la técnica la *Lista de Categorías de Clases Conceptuales*.

10.3. Clases conceptuales candidatas para el dominio de ventas

A partir del análisis de la Lista de Categorías de Clases Conceptuales y las frases nominales, se genera una lista de clases conceptuales candidatas del dominio. La lista está restringida a los requisitos y simplificaciones que se están estudiando actualmente —el escenario simplificado de *Procesar Venta*—.

<i>Registro</i>	<i>EspecificacionDelProducto</i>
<i>Articulo</i>	<i>LineaDeVenta</i>
<i>Tienda</i>	<i>Cajero</i>
<i>Venta</i>	<i>Cliente</i>
<i>Pago</i>	<i>Encargado</i>
<i>CatalogoDeProductos</i> ⁵	

No existe una lista “correcta”. Es una colección algo arbitraria de abstracciones y vocabulario del dominio que el modelador considera relevantes. En cualquier caso, siguiendo la estrategia de identificación, diferentes modeladores producirán listas similares.

⁵ N. del T.: Aunque siguiendo las convenciones para nombrar las clases no se utilizaría la preposición, en este caso al igual que en *EspecificacionDelProducto* y *LineaDeVenta*, se han utilizado para mejorar la legibilidad del texto.

Objetos de informes: ¿incluir el recibo en el modelo?

Un recibo es un informe de una venta y del pago, y una clase conceptual relativamente destacable del dominio, por tanto, ¿debería mostrarse en el modelo?

- A continuación presentamos algunos factores a tener en cuenta:
- Un recibo es un informe de una venta. En general, no es útil mostrar un informe de otra información en un modelo del dominio puesto que toda esta información se deriva de otras fuentes; duplica información que se encuentra en otras partes. Ésta es una razón para excluirlo.
 - Un recibo tiene un rol especial en término de las reglas del negocio: normalmente, confiere al portador del recibo, el derecho a devolver los artículos comprados. Ésta es una razón para mostrarlo en el modelo.

Puesto que la devolución de artículos no está siendo considerada en esta iteración, el *Recibo* será excluido. Durante la iteración que aborda el caso de uso *Gestionar Devoluciones*, estaría justificada su inclusión.

10.4. Guías para el modelado del negocio
Cómo hacer un modelo del dominio

Aplique los siguientes pasos para crear un modelo del dominio:

1. Liste las clases conceptuales candidatas, utilizando las técnicas de la Lista de Categorías de Clases Conceptuales y la identificación de frases nominales, relacionadas con los requisitos actuales en estudio.
 2. Represéntelos en un modelo del dominio.
 3. Añada las asociaciones necesarias para registrar las relaciones que hay que mantener en memoria (se discutirá en un capítulo siguiente).
 4. Añada los atributos necesarios para satisfacer los requisitos de información (se discutirá en un capítulo siguiente).

Un método útil auxiliar es aprender y copiar patrones de análisis, que se discutirán en un capítulo posterior.

Nombrar y modelar cosas: el cartógrafo

La estrategia del cartógrafo se aplica tanto a los mapas como a los modelos del dominio.

- Haga un modelo del dominio con el espíritu del modo de trabajo de los cartógrafos:

 - Utilice los nombres existentes en el territorio.
 - Excluya las características irrelevantes.
 - No añada cosas que no están ahí.

Un modelo del dominio es un tipo de mapa de conceptos o cosas de un dominio. Este espíritu destaca el rol analítico de un modelo del dominio, y sugiere lo siguiente:

- Un cartógrafo utiliza los nombres del territorio —no cambia los nombres de las ciudades en un mapa—. Para un modelo del dominio, significa que *utilice el vocabulario del dominio al nombrar los nombres de las clases conceptuales y los atributos*. Por ejemplo, si estamos desarrollando un modelo para una biblioteca, nombre al cliente como “*Prestatario*” o “*Socio*” —los términos utilizados por el personal de la biblioteca—.
- Un cartógrafo elimina las cosas de un mapa si no se consideran relevantes para el propósito del mapa; por ejemplo, la topografía o la población no es necesario que se muestren. Análogamente, un modelo de dominio podría excluir clases conceptuales del dominio del problema que no son pertinentes para los requisitos. Por ejemplo, podríamos excluir *Bolígrafo* y *BolsaPapel* de nuestro modelo del dominio (para el conjunto de requisitos actual) puesto que no tienen un rol relevante obvio.
- Un cartógrafo no muestra cosas que no estén ahí, como montañas que no existen. Igualmente, el modelo del dominio debería excluir cosas que *no* se encuentran en el dominio del problema que se está estudiando.

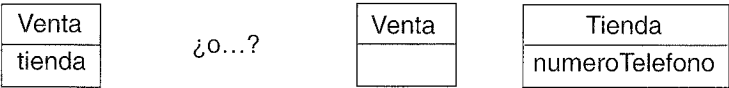
El principio también se conoce como la estrategia *Utilice el Vocabulario del Dominio* [Coad95].

Error típico en la identificación de las clases conceptuales

Quizás el error más típico al crear un modelo del dominio es representar algo como un atributo cuando debería haber sido un concepto. Una regla empírica para ayudar a prevenir este error es:

Si no consideramos alguna clase conceptual X que sea un número o texto en el mundo real, X es probablemente una clase conceptual, no un atributo.

Como ejemplo, ¿debería ser la *tienda* un atributo de *Venta*, o una clase conceptual separada *Tienda*?



En el mundo real, una tienda no se considera un número o un texto —el término sugiere una entidad legal, una organización, y algo que ocupa espacio—. Por tanto, *Tienda* debe ser un concepto.



Otro ejemplo, considere el dominio de las reservas de vuelos. ¿Debería ser el *destino* un atributo de *Vuelo*, o una clase conceptual aparte, *Aeropuerto*?

En el mundo real, un aeropuerto de destino no se considera un número o un texto —es una cosa grande que ocupa espacio—. Por tanto, *Aeropuerto* debería ser un concepto.

En caso de duda, considérela un concepto separado. Los atributos deberían ser bastante raros en un modelo del dominio.

10.5. Resolución de clases conceptuales similares: Registro vs. “TPDV”

TPDV son las siglas de terminal de punto de venta. En el lenguaje informático, un terminal es cualquier dispositivo de un sistema, como un PC cliente, un PDA en red inalámbrico, etcétera. En los primeros tiempos, mucho antes a los TPDVs, una tienda mantenía un *registro* —un libro en el que se apuntaban las ventas y pagos—. Con el tiempo, esto se automatizó en “cajas registradoras” mecánicas. Hoy en día, un TPDV desempeña el rol del registro (ver Figura 10.6).

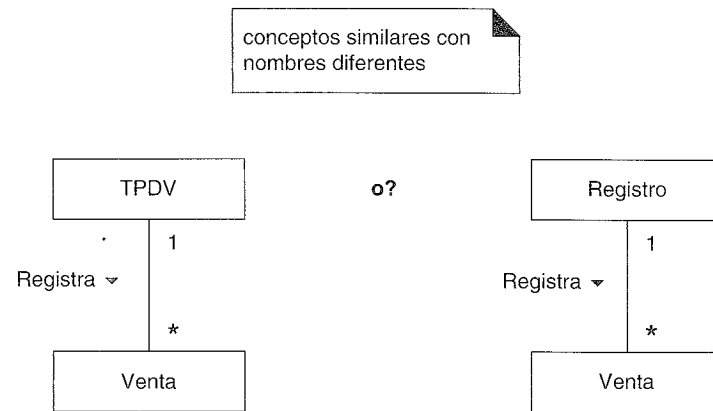


Figura 10.6. TPDV y Registro son clases conceptuales similares.

Un registro es una cosa que recoge las ventas y pagos, pero esto es un TPDV. Sin embargo, el término *registro* parece algo más abstracto y menos orientado a la implementación que un *TPDV*. Por tanto, en el modelo del dominio, ¿debería utilizarse el símbolo *Registro* en lugar de *TPDV*?

Primero, como regla empírica, un modelo del dominio no es absolutamente correcto o equivocado, sino más o menos útil; es una herramienta de comunicación.

Por el principio del cartógrafo, “*TPDV*” es un término familiar en el territorio, de manera que es un símbolo útil desde el punto de vista de la familiaridad y la comunicación. Por el objetivo de la creación de modelos que representan abstracciones y son indepen-

dientes de la implementación, *Registro* es atractivo y útil⁶. Se podría considerar *Registro* justamente, para representar tanto la clase conceptual de un sitio que registra las ventas, como una abstracción de varias clases de terminales, como un TPDV.

Ambas elecciones tienen valor; en este caso de estudio se ha elegido *Registro* algo arbitrariamente, pero el personal involucrado también habría entendido *TPDV*.

10.6. Modelado del mundo *irreal*

Algunos sistemas software son para dominios que encuentran muy poca analogía con dominios naturales o de negocios; el software para las telecomunicaciones es un ejemplo. Todavía es posible crear un modelo del dominio en estos dominios, pero requiere un alto grado de abstracción y olvidarse de diseños familiares.

Por ejemplo, algunas de las clases conceptuales candidatas relacionadas con conmutadores de telecomunicaciones son: *Mensaje*, *Conexion*, *Puerto*, *Dialogo*, *Ruta*, *Protocolo*.

10.7. Clases conceptuales de especificación o descripción

La siguiente discusión podría, al principio, parecer relacionada con un tema raro y altamente especializado. Sin embargo, prueba que la necesidad de las clases conceptuales de especificación (como se definirán) es habitual en muchos modelos del dominio. Por tanto, se destaca.

Asuma lo siguiente:

- Una instancia de un *Articulo* representa un objeto físico de una tienda; como tal, podría incluso tener un número de serie.
- Un *Articulo* tiene una descripción, precio, e identificador del artículo (*articuloID*), que no se recogen en ningún otro sitio.
- Todo el mundo que trabaja en la tienda tiene amnesia.
- Cada vez que se vende un artículo físico real, se elimina una instancia de *Articulo* software correspondiente, del “terreno del software”.

Con estas suposiciones, ¿qué ocurre en el siguiente escenario?

Existe una fuerte demanda de las nuevas hamburguesas vegetales que han tenido mucho éxito —ObjetoHamburguesa—. La tienda las vende todas, lo que implica que se eliminan de la memoria del ordenador todas las instancias de *Articulo* de los ObjetoHamburguesa.

⁶ Nótese que en los primeros tiempos, un *registro* era únicamente una posible implementación del modo de registrar las ventas. El término ha adquirido un significado generalizado a lo largo del tiempo.

Ahora, aquí está lo esencial del problema: Si alguien pregunta: “¿Cuánto cuestan los ObjetosHamburguesa?”, nadie puede contestar, porque el precio se encontraba en las instancias inventariadas, que se eliminaron cuando se vendieron.

Nótese también que el modelo actual, si se implementa en el software tal y como se describe, contiene datos duplicados y es ineficiente en cuanto al espacio, puesto que la descripción, el precio y el ID, se duplican en cada instancia de *Articulo* del mismo producto.

La necesidad de especificación o descripción de las clases conceptuales

El problema anterior ilustra la necesidad de conceptos de objetos que sean especificaciones o descripciones de otras cosas. Para solucionar el problema del *Articulo*, lo que se necesita es una clase conceptual *EspecificacionDelProducto* (o *EspecificacionDelArticulo*, *DescripcionDelProducto*, ...) que recoge la información sobre los artículos. Una *EspecificacionDelProducto* no representa un *Articulo*, sino una descripción de la información sobre los artículos. Nótese que incluso si todos los elementos inventariados se venden, y sus correspondientes instancias software de *Articulo* se eliminan, permanece todavía la *EspecificacionDelProducto*.

Los objetos de descripción o especificación están fuertemente relacionados con las cosas que describen. En un modelo del dominio, es típico establecer que una *EspecificacionDeX Describe un X* (ver Figura 10.7).

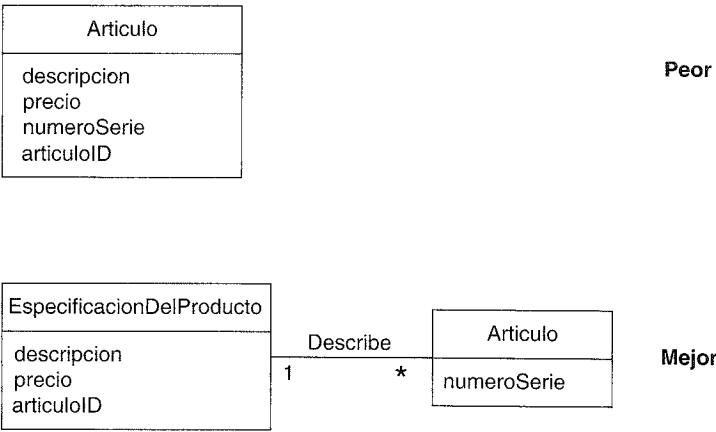


Figura 10.7. Especificaciones o descripciones sobre otras cosas. El “*” significa una multiplicidad de “muchos”. Indica que una *EspecificacionDelProducto* podría describir a muchos (*) *Articulos*.

La necesidad de clases conceptuales de especificación es habitual en los dominios de ventas y productos. También es típica en la fabricación, donde se requiere una descripción de la cosa que se fabrica, que es distinto de la cosa en sí misma. Se ha dedicado tiempo y espacio a motivar las clases conceptuales de especificación porque son muy frecuentes; no es un concepto de modelado raro.

¿Cuándo se requieren las clases conceptuales de especificación?

Las siguientes guías sugieren cuándo utilizar las especificaciones:

Añada una clase conceptual de especificación o descripción (por ejemplo, *EspecificacionDelProducto*) cuando:

- Se necesita la descripción de un artículo o servicio, independiente de la existencia actual de algún ejemplo de esos artículos o servicios.
- La eliminación de instancias de las cosas que describen (por ejemplo, *Articulo*) dan como resultado una pérdida de información que necesita mantenerse, debido a la asociación incorrecta de información con la cosa eliminada.
- Reduce información redundante o duplicada.

Otro ejemplo de especificación

Como otro ejemplo, considere una compañía aérea que sufre un accidente catastrófico de uno de sus aviones. Asuma que se cancelan todos los vuelos durante seis meses, pendiente de que se complete la investigación. También asuma que, cuando se cancelan los vuelos, su correspondiente objeto software *Vuelo*, se elimina de la memoria del ordenador. Por tanto, después de la colisión, se eliminan todos los objetos software *Vuelo*.

Si el único registro de los aeropuertos de destino de los vuelos se encuentra en las instancias software *Vuelo*, que representa un vuelo específico en una fecha y hora concreta, ya no habrá un registro de las rutas de los vuelos de la compañía.

Para solucionar este problema, se necesita una *DescripcionDelVuelo* (o *EspecificacionDelVuelo*) que describe un vuelo y su ruta, incluso cuando no se ha planificado ningún vuelo concreto (ver Figura 10.8).

Descripción de servicios

Nótese que el ejemplo anterior trata sobre un servicio (un vuelo) en lugar de un artículo (como una hamburguesa vegetal). Es habitual que se necesiten descripciones de servicios o planes de servicios.

Como otro ejemplo, una compañía de telefonía móvil vende paquetes con denominaciones como “bronce”, “oro”, etcétera. Es necesario tener el concepto de descripción del paquete (un tipo de plan de servicio que describe las tarifas por minuto, acceso a Internet sin cable, el coste, etcétera) separado del concepto del paquete realmente vendido (como el “paquete oro vendido a Craig Larman el 1 de enero de 2002, a \$55 al mes”). El departamento de Marketing necesita definir y registrar este plan de servicios o *DescripcionDelPaqueteDeComunicacionesMoviles* antes de que se venda alguno.

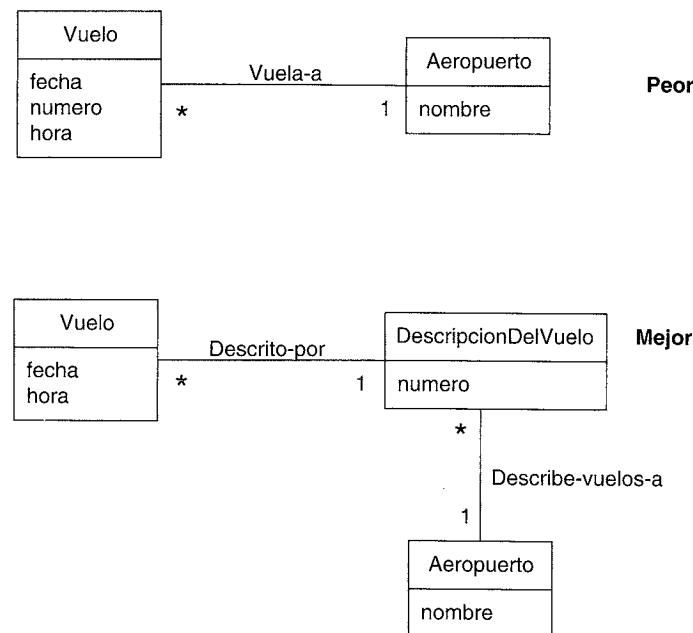


Figura 10.8. Especificaciones sobre otras cosas.

10.8. Notación UML, modelos y métodos: perspectivas múltiples

El UP define algo denominado Modelo del Dominio, que se representa con la notación UML. Sin embargo, no existe un término “Modelo del Dominio” en la documentación oficial de UML. Esto nos revela algo importante:

UML simplemente describe tipos de diagramas, como los diagramas de clases y los diagramas de secuencia. No superpone un método o perspectiva de modelado sobre ellos. Más bien, un proceso (como el UP) aplica la notación de la especificación de UML en el contexto de modelos definidos en el ámbito de una metodología.

Por ejemplo, la notación de los diagramas de clases de UML se puede utilizar para crear representaciones de las clases conceptuales del dominio (un modelo del dominio), clases software, tablas de una base de datos relacional, etcétera.

Por tanto, no debemos confundir la notación básica de los diagramas UML, con su aplicación para visualizar distintos tipos de modelos definidos por los metodólogos (ver Figura 10.9). Este punto es aplicable no sólo a los diagramas de clases UML, sino a la mayoría de la notación UML.

Como otro ejemplo de los diagramas de UML que se interpretan de manera diferente en modelos distintos, los diagramas de secuencia UML se pueden utilizar para representar el paso de mensajes entre los objetos software (como en el Modelo de Diseño del UP), o la interacción entre personas y grupos en el mundo real (como en el Modelo de Objetos del Negocio del UP).

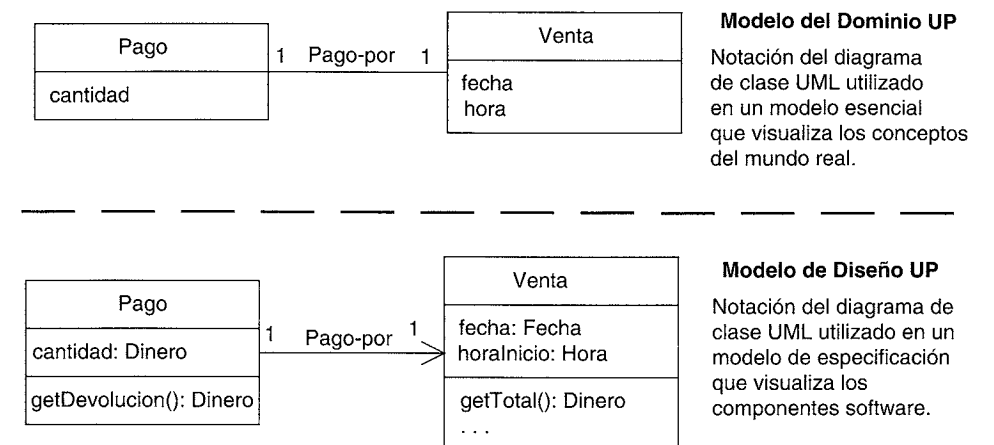


Figura 10.9. La notación de la especificación UML se puede aplicar en diferentes perspectivas y modelos definidos por un proceso o método.

Esta idea se pone de relieve en el método orientado a objetos Syntropy [CD94], y Martin Fowler la reitera en *UML Distilled* [FS00]. Es decir, la misma notación basada en diagramas se puede utilizar en tres perspectivas y tipos de modelos:

1. **Perspectiva esencial o conceptual:** se interpreta que los diagramas describen cosas del mundo real o de un dominio de interés.
2. **Perspectiva de especificación:** se interpreta que los diagramas (utilizando la misma notación de los modelos esenciales) describen abstracciones software o componentes con especificaciones e interfaces, pero no comprometidas a ninguna implementación en particular (por ejemplo, no específicamente una clase C# o Java).
3. **Perspectiva de implementación:** se interpreta que los diagramas (utilizando la misma notación de los modelos esenciales) describen implementaciones software con una tecnología y lenguaje particular (como Java).

Superposición de terminología: UML vs. métodos

En la especificación UML, las cajas rectangulares mostradas en la Figura 10.9 se denominan **clases**, pero nótese que en UML, este término abarca una variedad de fenómenos —cosas físicas, cosas del software, eventos, etcétera⁷—. Un proceso o método superpondrá una terminología alternativa sobre UML. Por ejemplo, en el UP, cuando las cajas de UML se dibujan en el Modelo del Dominio, podrían llamarse **conceptos del dominio** o **clases conceptuales**; el Modelo del Dominio ofrece una perspectiva conceptual. En el UP, cuando las cajas UML se dibujan en el Modelo de Diseño, se denominan oficialmente **clases de diseño**: el Modelo de Diseño ofrece una perspectiva de especificación o implementación, como quiere el modelador.

⁷ Una clase UML es un caso especial de un elemento de modelado UML muy general el **clasificador** —algo con características estructurales y/o comportamiento, incluyendo clases, actores, interfaces y casos de uso—.

Independientemente de la definición, la cuestión importante es que es útil distinguir entre la perspectiva de un analista que mira conceptos del mundo real tal como una venta (una perspectiva conceptual), y los diseñadores de software que especifican componentes software como la clase software *Venta* (una perspectiva de especificación o implementación).

UML se puede utilizar para ilustrar ambas perspectivas con una notación y perspectiva muy similar, así que es muy importante tener en cuenta qué perspectiva se está tomando.

Para mantener las cosas claras, este libro utilizará los términos relacionados con las clases de la siguiente forma, que es consistente con UML y el UP:

- **Clase conceptual:** concepto o cosa del mundo real. Una perspectiva conceptual o esencial. El Modelo del Dominio del UP contiene clases conceptuales.
- **Clase software:** una clase que representa una perspectiva de especificación o implementación de un componente software, independientemente del proceso o método.
- **Clase de diseño:** un miembro del Modelo de Diseño del UP. Es un sinónimo de clase software, pero por alguna razón deseo resaltar que es una clase del Modelo de Diseño. El UP permite que una clase de diseño tenga perspectiva de especificación o implementación, según desee el modelador.
- **Clase de implementación:** una clase implementada en un lenguaje orientado a objetos como Java.
- **Clase:** como en UML, el término general que representa o una cosa del mundo real (una clase conceptual) o del software (una clase software).

10.9. Reducción del salto en la representación

Por favor, mire atentamente la Figura 10.10. ¿Por qué los libros y educadores que presentan el diseño de objetos común sólo muestran el uso de clases software cuyos nombres reflejan el vocabulario del dominio? ¿Por qué eligen un nombre de clase software como *Venta*, y qué hace una *Venta*?

Simplemente, eligiendo nombres que reflejan el vocabulario del dominio (*Venta*) se favorece la rápida comprensión y proporciona una pista acerca de lo que se espera del trozo de código de la clase software *Venta*. Tenemos un modelo mental del dominio en cuestión (por ejemplo, una tienda que vende cosas). En el mundo real, sabemos que una venta tiene una fecha. En consecuencia, si creamos una clase Java llamada *Venta*, y le damos la responsabilidad de conocer sobre una venta real y su fecha, entonces la clase Java *Venta* se corresponde de alguna manera con nuestro modelo mental del dominio real; es decir, apela a nuestra “intuición” del dominio.

El Modelo del Dominio proporciona un diccionario visual del vocabulario y conceptos del dominio a partir de los cuales nos inspiramos para nombrar algunas cosas del diseño software.

Esto se relaciona con el tema del **salto de la representación** o salto semántico —el salto entre nuestro modelo mental del dominio y su representación en el software—.

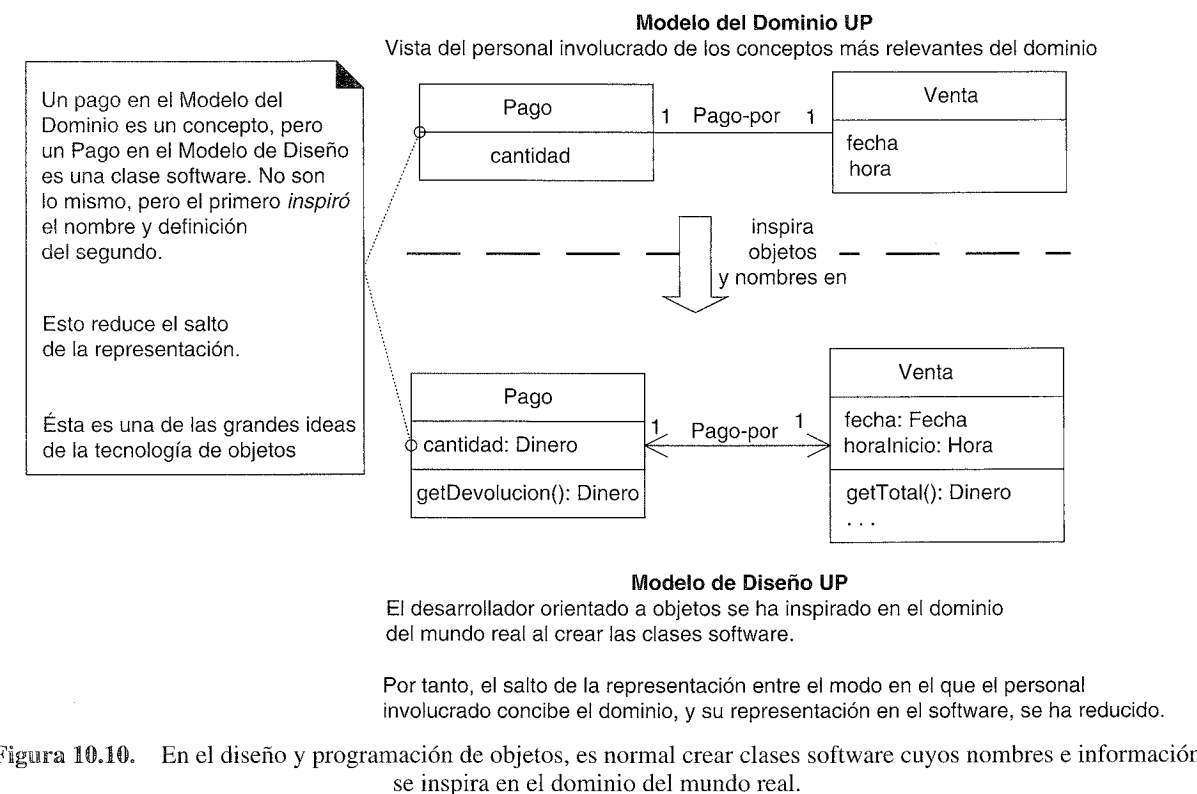


Figura 10.10. En el diseño y programación de objetos, es normal crear clases software cuyos nombres e información se inspira en el dominio del mundo real.

En un extremo, podríamos programar directamente la aplicación de PDV NuevaE-*ra* en código binario para invocar el conjunto de instrucciones del procesador. Entendemos que el salto en las representaciones es enorme, y existirá en el software un coste real —aunque difícil de cuantificar— con un salto en la representación tan grande, porque es difícil de entender y relacionar con el dominio del problema. Cercano al otro extremo del espectro, están las tecnologías de objetos que nos permiten partir el código en clases cuyos nombres reflejan el tipo de partición que percibimos en el dominio. En el mundo real percibimos una “parte” (o hecho) denominada venta, luego en el terreno del software tenemos una clase software denominada *Venta*. Esta estrecha correspondencia uno-a-uno entre el vocabulario del dominio y nuestro vocabulario del software y sus particiones reduce el salto de la representación. Esto acelera la comprensión del código existente (porque funciona de la manera que esperamos, conociendo el dominio) y sugiere formas “naturales” para extender el código que se corresponden análogamente con el dominio, o apelan a nuestras intuiciones del dominio. Diciéndolo sencillamente, el modelo software nos recuerda al modelo mental o conceptual, y funciona de manera predecible.

Los modelos software proporcionan una ventaja práctica al reducir el salto de la representación. La mayoría de los ingenieros del software saben que es verdad, incluso si es difícil de cuantificar. De hecho, una prueba de esto es que algunos de los programadores Java, de forma deliberada complican el código fuente para que sea difícil hacer ingeniería inversa a partir de los bytecodes, cambiando los nombres de las clases y los métodos Java de manera que sean ininteligibles y, por tanto, ya no evoquen nuestra visión del dominio, aunque las estructuras de control y datos no se cambian.

Por supuesto, la tecnología de objetos es valiosa también porque puede favorecer el diseño de sistemas elegantes, débilmente acoplados, que pueden crecer y ampliarse fácilmente, como veremos en lo que queda del libro. Una reducción del salto en la representación es útil, pero se puede sostener que es secundario a la ventaja que ofrecen los objetos de facilitar los cambios y extensiones, y el soporte que ofrecen para el manejo y ocultación de la complejidad.

10.10. Ejemplo: el Modelo del Dominio del PDV NuevaEra

La lista de clases conceptuales generadas para el dominio del PDV NuevaEra se podría representar gráficamente (ver Figura 10.11) para mostrar el comienzo del Modelo del Dominio.

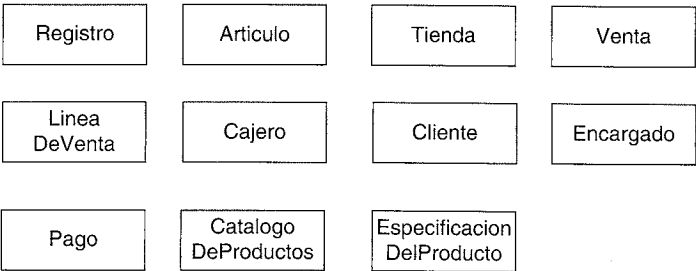


Figura 10.11. Modelo del Dominio inicial.

Las consideraciones acerca de los atributos y asociaciones del Modelo del Dominio se posponen a capítulos siguientes.

10.11. Modelos del Dominio en el UP

Como se sugiere en el ejemplo de la Tabla 10.2, un Modelo del Dominio, normalmente, se inicia y completa en la elaboración.

Inicio

Los modelos del dominio no se incentivan fuertemente en la fase de inicio, puesto que el propósito del inicio no es llevar a cabo un estudio serio, sino decidir si merece la pena un estudio más profundo en el proyecto, en una fase de elaboración.

Elaboración

El Modelo del Dominio se crea sobre todo durante las iteraciones de la elaboración, cuando la necesidad más importante es entender los conceptos relevantes y trasladar algunos a clases software durante el trabajo de diseño.

Tabla 10.2. Muestra de los artefactos UP y evolución temporal. c – comenzar; r - refinar.

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c	r	
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Aunque, irónicamente, se dedicarán un número significativo de páginas a explicar el modelado de los objetos del dominio, en manos experimentadas el desarrollo de un modelo del dominio (parcial, desarrollado incrementalmente) en cada iteración debería durar sólo unas pocas horas. Esto se acorta mediante el uso de patrones de análisis predefinidos.

El Modelo de Objetos del Negocio del UP vs. el Modelo del Dominio

El Modelo del Dominio del UP es una variación oficial del menos común Modelo de Objetos del Negocio del UP (BOM, *Business Object Model*). El BOM del UP —no confundir con el modo en el que otras personas o métodos podrían definir un BOM, que es un término ampliamente utilizado con significados diferentes— es un tipo de modelo de empresa utilizado para describir el negocio completo. Podría utilizarse al llevar a cabo la ingeniería o reingeniería de procesos del negocio, independiente de cualquier aplicación software (como el PDV NuevaEra). Citando textualmente:

[El BOM del UP] sirve como abstracción del modo en el que los trabajadores y las entidades del negocio necesitan relacionarse y cómo necesitan colaborar para llevar a cabo el negocio. [RUP]

El BOM se representa con varios diagramas diferentes (clase, actividad y secuencia) que muestran cómo funciona (o debería funcionar) toda la empresa. Es más útil si se realiza ingeniería de procesos de negocio por toda la empresa, pero ésta es una actividad menos común que la creación de una única aplicación software.

En consecuencia, el UP define el Modelo del Dominio como un artefacto subconjunto o una especialización del BOM, que se crea normalmente. Citando textualmente:

Puedes elegir desarrollar un modelo de objetos del negocio “incompleto”, enfocado a explicar “cosas” y productos importantes para un dominio. ... A menudo se hace referencia a éste como modelo del dominio. [RUP]

10.12. Lecturas adicionales

Object-Oriented Methods: A Foundation de Odell proporciona una sólida introducción al modelado del dominio conceptual. También resulta útil *Design Object Systems* de Cook and Daniel.

Analysis Patterns de Fowler ofrece importantes patrones de los modelos del dominio, y se recomienda sin ninguna duda. Otro libro bueno, que describe patrones de los modelos del dominio es *Data Model Patterns: Conventions of Thought* de Hay. Los consejos de los expertos en el modelado de datos, que entienden la distinción entre los modelos conceptuales puros y los modelos de los esquemas de bases de datos, pueden ser muy útiles para el modelado de los objetos del dominio.

Java Modeling in Color with UML [CDL99] contiene más consejos relevantes sobre el modelado del dominio de lo que sugiere el título. El autor identifica patrones comunes en los tipos relacionados y sus asociaciones; el aspecto de color es realmente una visualización de las categorías típicas de estos tipos, como *descripción* (azul), *roles* (amarillo), y *momento-intervalos* (rosa). El color se utiliza para ayudar a ver los patrones.

Desde el trabajo original de Abbot, el análisis lingüístico ha adquirido técnicas más sofisticadas para el análisis orientado a objetos, generalmente denominado modelado del lenguaje natural, o una variante. Véase [Moreno97] como un ejemplo.

10.13. Artefactos del UP

La influencia entre los artefactos, destacando el Modelo del Dominio, se muestra en la Figura 10.12.

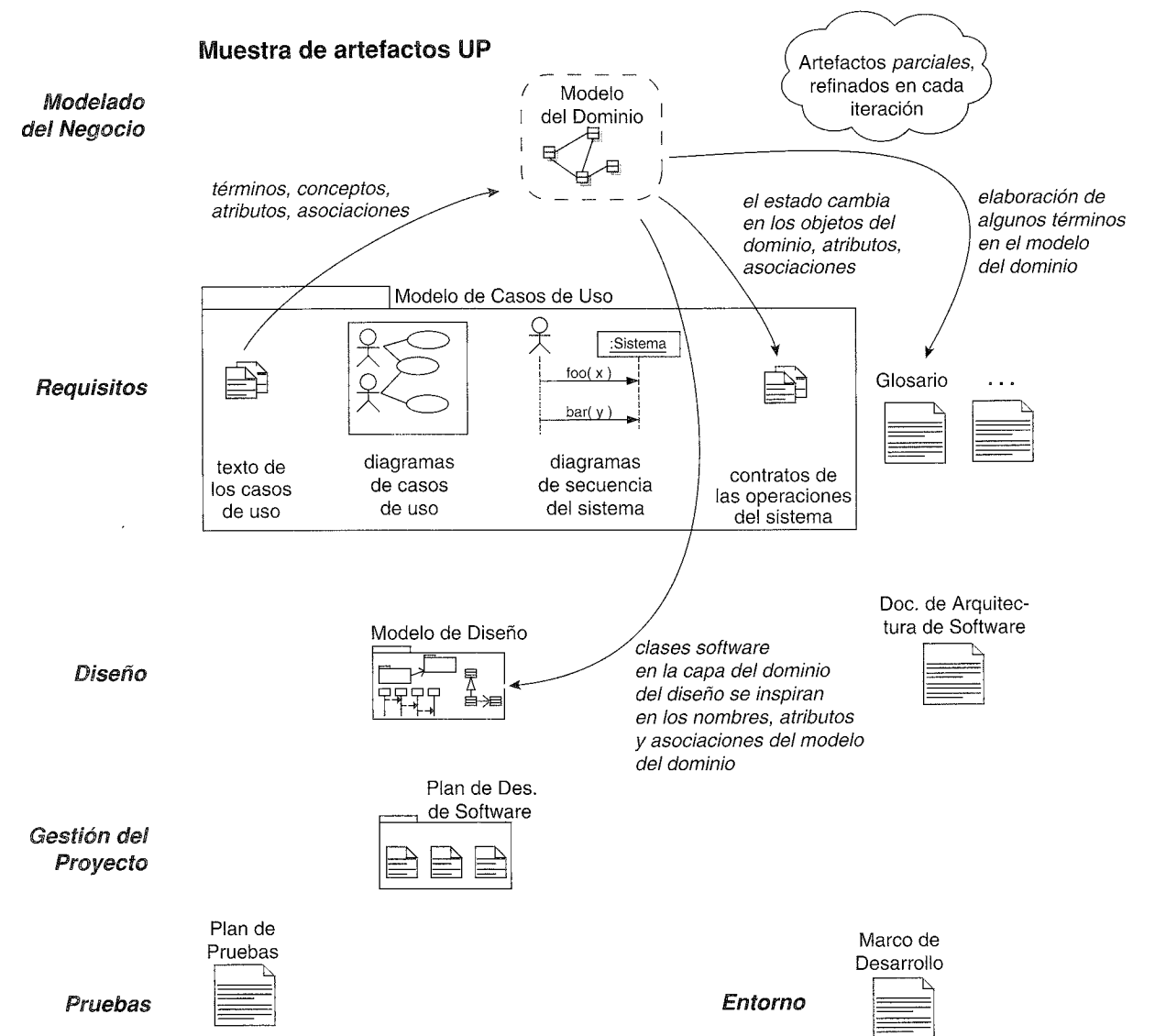


Figura 10.12. Muestra de la influencia entre los artefactos UP.

MODELO DEL DOMINIO: AÑADIR ASOCIACIONES

Objetivos

- Identificar las asociaciones del modelo del dominio.
 - Distinguir entre las asociaciones necesito-conocer y sólo-comprensión.
-

Introducción

Resulta útil identificar aquellas asociaciones entre clases conceptuales que son necesarias para satisfacer los requisitos de información de los escenarios actuales que se están desarrollando, y que ayudan a entender el modelo del dominio. Este capítulo explora la identificación de las asociaciones adecuadas, y añade las asociaciones al modelo del dominio del caso de estudio NuevaEra.

11.1. Asociaciones

Una **asociación** es una relación entre tipos (o más concretamente, instancias de estos tipos) que indica alguna conexión significativa e interesante (ver Figura 11.1).

En UML, las asociaciones se definen como “la relación semántica entre dos o más clasificadores que implica conexiones entre sus instancias”.

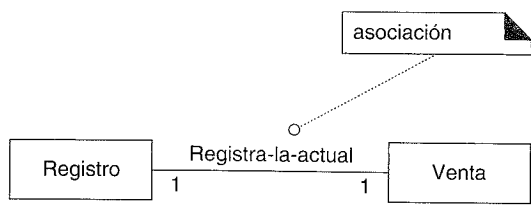


Figura 11.1. Asociaciones.

Criterio para asociaciones útiles

Las asociaciones que merece la pena registrar, normalmente, implican conocimiento de una relación que es necesario conservar durante algún tiempo —podría ser milisegundos o años, dependiendo del contexto—. En otras palabras, ¿entre qué objetos necesitamos mantener en memoria una relación? Por ejemplo, ¿necesitamos recordar qué instancias de *LineaDeVenta* están asociadas con una instancia de *Venta*? Sin ninguna duda, de otra manera no sería posible reconstruir una venta, imprimir un recibo o calcular el total de la venta.

Considere la inclusión de las siguientes asociaciones en un modelo del dominio:

- Asociaciones de las que es necesario conservar el conocimiento de la relación durante algún tiempo (asociaciones “necesito-conocer”).
- Asociaciones derivadas de la Lista de Asociaciones Comunes.

Por el contrario, ¿necesitamos recordar una relación entre la *Venta* actual y un *Encargado*? No, los requisitos no dan a entender que se necesite ninguna relación de este tipo. No es incorrecto mostrar una relación entre una *Venta* y un *Encargado*, pero no es convincente o útil en el contexto de nuestros requisitos.

Éste es un punto importante. En un modelo del dominio con *n* clases del dominio diferentes, pueden existir *n*(*n*-1) asociaciones entre diferentes clases conceptuales —un número potencialmente grande—. Muchas líneas en un diagrama añadirán “ruido visual” y lo hará menos comprensible. Por tanto, sea cuidadoso al añadir líneas de asociación. Utilice como criterio las guías que se sugieren en este capítulo.

11.2. Notación de las asociaciones en UML

Una asociación se representa como una línea entre clases con un nombre de asociación. La asociación es inherentemente bidireccional, lo que significa que desde las instancias de cualquiera de las dos clases, es posible el recorrido lógico hacia la otra.

Este recorrido es puramente abstracto; *no* se trata de una sentencia sobre conexiones entre entidades software.

Los extremos de la asociación podrían contener una expresión de multiplicidad que indica la relación numérica entre las instancias de las clases.

Una “flecha de dirección de lectura” opcional indica la dirección de la lectura del nombre de la asociación; *no* indica la dirección de la visibilidad o navegación.

Si no está presente, la convención es leer la asociación de izquierda a derecha o de arriba hacia abajo, aunque no es una regla de UML (ver Figura 11.2).

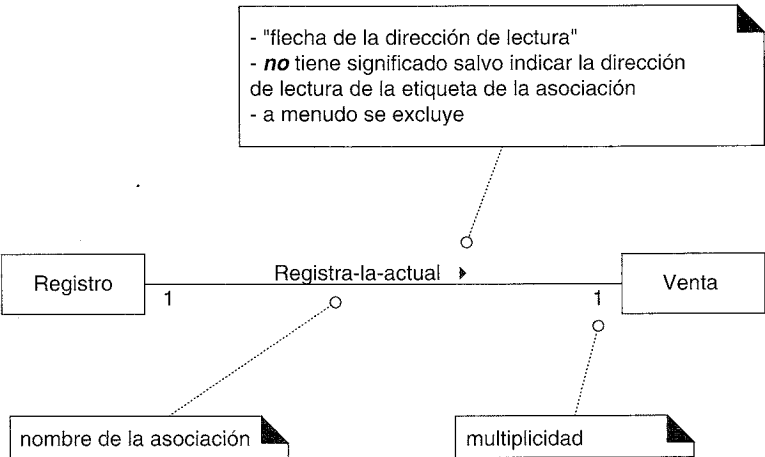


Figura 11.2. Notación UML para las asociaciones.

La flecha de dirección de lectura no tiene significado en términos del modelo; sólo es una ayuda para el lector del diagrama.

11.3. Localización de las asociaciones—lista de asociaciones comunes

Comience la inclusión de asociaciones utilizando la lista de la Tabla 11.1.

Contiene categorías comunes que, normalmente, merece la pena tener en cuenta. Los ejemplos se han extraído de los dominios de ventas y reservas de vuelos.

Tabla 11.1. Lista de asociaciones comunes.

Categoría	Ejemplos
A es una parte física de B	Cajon-Registro (o más concretamente, TPDV) Ala-Avion
A es una parte lógica de B	LineaDeVenta-Venta EtapaVuelo-RutaVuelo
A está contenido físicamente en B	Registo-Tienda, Artículo-Estanteria Pasajero-Avion
A está contenido lógicamente en B	DescripcionDelArticulo-Catalogo Vuelo-PlanificacionVuelo

continúa

Tabla 11.1. Lista de asociaciones comunes. (Continuación)

Categoría	Ejemplos
A es una descripción de B	DescripcionDelArticulo-Articulo DescripcionDelVuelo-Vuelo
A es un línea de una transacción o informe de B	LineaDeVenta-Venta TrabajoMantenimiento-RegistroDe-Mantenimiento
A se conoce/registra/recoge/informa/cap-tura en B	Venta-Registro Reserva-ListaPasajeros
A es miembro de B	Cajero-Tienda Piloto-CompañíaAerea
A es una subunidad organizativa de B	Departamento-Tienda Mantenimiento-CompañíaAerea
A utiliza o gestiona B	Cajero-Registro Piloto-Avion
A se comunica con B	Cliente-Cajero AgenteDeReservas-Pasajero
A está relacionado con una transacción B	Cliente-Pago Pasajero-Billete
A es una transacción relacionada con otra transacción B	Pago-Venta Reserva-Cancelacion
A está al lado de B	LineaDeVenta-LineaDeVenta Ciudad-Ciudad
A es propiedad de B	Registro-Tienda Avion-CompañíaAerea
A es un evento relacionado con B	Venta-Cliente, Venta-Tienda Salida-Vuelo

Asociaciones de prioridad alta

A continuación, presentamos algunas de las categorías de asociaciones de prioridad alta que son, invariablemente, útiles incluirlas en un modelo del dominio:

- A es una *parte lógica o física* de B.
- A está *contenida física o lógicamente* en B.
- A se *registra* en B.

11.4. Guías para las asociaciones

- Céntrese en aquellas asociaciones para las que se necesita conservar el conocimiento de la relación durante algún tiempo (asociaciones “necesito-conocer”).
- Es más importante identificar *clases conceptuales* que identificar asociaciones.

- Demasiadas asociaciones tienden a confundir un modelo del dominio en lugar de aclararlo. Su descubrimiento puede llevar tiempo, con beneficio marginal.
- Evite mostrar asociaciones redundantes o derivadas.

11.5. Roles

Cada extremo de una asociación se denomina **rol**. Los roles pueden tener opcionalmente:

- Nombre.
- Expresión de multiplicidad.
- Navegabilidad.

La multiplicidad se presenta a continuación, y las otras dos características se discutirán en capítulos posteriores.

Multiplicidad

La **multiplicidad** define cuántas instancias de una clase A pueden asociarse con una instancia de una clase B (ver Figura 11.3).

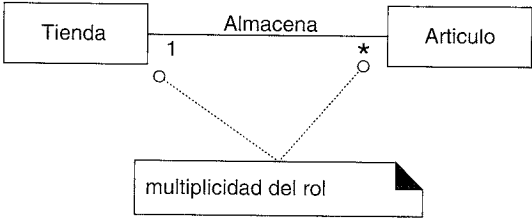


Figura 11.3. Multiplicidad de una asociación.

Por ejemplo, una instancia individual de una *Tienda* puede asociarse con “muchas” (cero o más, indicado por el “*”) instancias de *Artículo*.

En la Figura 11.4 se muestran algunos ejemplos de expresiones de multiplicidad.

El valor de la multiplicidad indica cuántas instancias se puede asociar legalmente con otra, en un momento concreto, en lugar de a lo largo de un periodo de tiempo. Por ejemplo, es posible que un coche usado pudiera a lo largo del tiempo ser vendido repetidamente a comerciantes de coches usados. Pero en un momento concreto, el coche sólo es *Abastecido-por* un comerciante. El coche no es *Abastecido-por* muchos comerciantes en un momento concreto. Análogamente, en países con leyes monógamas, una persona puede estar *Casado-con* sólo una persona en un momento dado, aunque a lo largo del tiempo, puedan casarse con muchas personas.

El valor de la multiplicidad depende de nuestros intereses como modeladores y desarrolladores de software, porque pone de manifiesto una restricción de diseño que será (o podrá ser) reflejada en el software. La Figura 11.5 muestra un ejemplo y explicación.